Финансовый университет при Правительстве Российской Федерации

Факультет информационных технологий и анализа больших данных

Направление подготовки 10.03.01 «Информационная безопасность»

ОТЧЁТ

по результатам анализа защищённости веб-приложения
OWASP Juice Shop

(поиск, эксплуатация и устранение уязвимостей)

Выполнил: студент группы ИБ24-5
Шишкин Кирилл Владимирович

Москва 2026

СОДЕРЖАНИЕ

1. Введение.....6
2. Этап разведки.....7
3. Обнаруженные уязвимости.....13
3.1. Уязвимость №1. SQL-инъекция в форме входа (обход аутентификации) 13
3.1.1. Как нашли.....13
3.1.2. Как использовать.....14
3.1.3. Как исправить.....14
3.2. Уязвимость №2. UNION-based SQL-инъекция в поиске товаров (выгрузка БД).....16
3.2.1. Как нашли.....16
3.2.2. Как использовать.....16
3.2.3. Как исправить.....17
3.3. Уязвимость №3. NoSQL-инъекция (массовое изменение отзывов) 18
3.3.1. Как нашли.....18
3.3.2. Как использовать.....18
3.3.3. Как исправить.....19
3.4. Уязвимость №4. DOM-based XSS в поле поиска.....20
3.4.1. Как нашли.....20
3.4.2. Как использовать.....20
3.4.3. Как исправить.....20
3.5. Уязвимость №5. Отражённый XSS в параметре отслеживания заказа 22
3.5.1. Как нашли.....22
3.5.2. Как использовать.....22
3.5.3. Как исправить.....22
3.6. Уязвимость №6. Хранимый (persisted) XSS через HTTP-заголовок23
3.6.1. Как нашли.....23
3.6.2. Как использовать.....24
3.6.3. Как исправить.....24
3.7. Уязвимость №7. Нарушение контроля доступа: доступ к разделу администрирования.....25
3.7.1. Как нашли.....26
3.7.2. Как использовать.....26
3.7.3. Как исправить.....26

3.8. Уязвимость №8. IDOR: доступ к чужой корзине....28
3.8.1. Как нашли....28
3.8.2. Как использовать....28
3.8.3. Как исправить....28
3.9. Уязвимость №9. Подделка отзыва от имени другого пользователя 29
3.9.1. Как нашли....30
3.9.2. Как использовать....30
3.9.3. Как исправить....30
3.10. Уязвимость №10. Mass Assignment: регистрация с правами администратора....31
3.10.1. Как нашли....31
3.10.2. Как использовать....31
3.10.3. Как исправить....32
3.11. Уязвимость №11. Сброс пароля через слабый секретный вопрос. 33
3.11.1. Как нашли....33
3.11.2. Как использовать....33
3.11.3. Как исправить....33
3.12. Уязвимость №12. Отсутствие защиты от перебора (brute force) при входе....35
3.12.1. Как нашли....35
3.12.2. Как использовать....35
3.12.3. Как исправить....35
3.13. Уязвимость №13. Раскрытие данных: доступ к каталогу /ftp.....36
3.13.1. Как нашли....37
3.13.2. Как использовать....37
3.13.3. Как исправить....37
3.14. Уязвимость №14. Обход фильтра расширений через Poison Null Byte (path traversal)....38
3.14.1. Как нашли....38
3.14.2. Как использовать....38
3.14.3. Как исправить....39
3.15. Уязвимость №15. Раскрытие конфиденциального документа о поглощениях....40
3.15.1. Как нашли....40
3.15.2. Как использовать....40
3.15.3. Как исправить....40

3.16. Уязвимость №16. Доступ к журналам сервера (/support/logs).....41
3.16.1. Как нашли.....42
3.16.2. Как использовать.....42
3.16.3. Как исправить.....42
3.17. Уязвимость №17. XXE: чтение файлов через загрузку XML.....43
3.17.1. Как нашли.....43
3.17.2. Как использовать.....43
3.17.3. Как исправить.....44
3.18. Уязвимость №18. SSRF через подмену URL профильного изображения 45
3.18.1. Как нашли.....45
3.18.2. Как использовать.....45
3.18.3. Как исправить.....45
3.19. Уязвимость №19. Подделка JWT через алгоритм none / слабый секрет 47
3.19.1. Как нашли.....47
3.19.2. Как использовать.....47
3.19.3. Как исправить.....47
3.20. Уязвимость №20. Слабое хеширование паролей (MD5).....48
3.20.1. Как нашли.....49
3.20.2. Как использовать.....49
3.20.3. Как исправить.....49
3.21. Уязвимость №21. Небезопасная десериализация / выполнение кода 50
3.21.1. Как нашли.....51
3.21.2. Как использовать.....51
3.21.3. Как исправить.....51
3.22. Уязвимость №22. Уязвимые и устаревшие компоненты (перезапись legal.md).....52
3.22.1. Как нашли.....53
3.22.2. Как использовать.....53
3.22.3. Как исправить.....53
3.23. Уязвимость №23. Обход защиты от автоматизации (10+ отзывов за 10 секунд).....54
3.23.1. Как нашли.....55
3.23.2. Как использовать.....55
3.23.3. Как исправить.....55

3.24. Уязвимость №24. Отсутствие/слабость заголовков безопасности (CSP) 56
3.24.1. Как нашли 56
3.24.2. Как использовать 56
3.24.3. Как исправить 56
3.25. Уязвимость №25. Слабая криптография купонов (предсказуемые скидки) 58
3.25.1. Как нашли 58
3.25.2. Как использовать 58
3.25.3. Как исправить 58
3.26. Уязвимость №26. Безопасность через сокрытие (утечка до анонса) 59
3.26.1. Как нашли 60
3.26.2. Как использовать 60
3.26.3. Как исправить 60
3.27. Уязвимость №27. Небезопасная загрузка файлов (тип и размер) .. 61
3.27.1. Как нашли 61
3.27.2. Как использовать 61
3.27.3. Как исправить 61
4. Заключение 63
5. Список литературы 63

1. Введение

OWASP Juice Shop — умышленно уязвимое веб-приложение под эгидой OWASP (Open Worldwide Application Security Project), предназначенное для обучения тестированию защищённости. Приложение построено на стеке Node.js, Express и Angular, использует ORM Sequelize поверх СУБД SQLite и MongoDB-совместимое хранилище MarsDB, и содержит набор преднамеренно внедрённых уязвимостей, покрывающих рейтинг OWASP Top 10 2021.

Теоретическую основу составляют материалы проекта OWASP Juice Shop[1], официальное руководство Pwning OWASP Juice Shop[2] и классификация OWASP Top 10[3].

Цель работы — провести анализ защищённости приложения: выполнить разведку, обнаружить максимально возможное число уязвимостей, продемонстрировать их эксплуатацию и предложить способы устранения. Все действия выполнялись на локальном учебном стенде: серверная часть Juice Shop развёрнута на хосте Windows 10, атакующая машина — Kali Linux. Тестирование умышленно уязвимого приложения в изолированной среде легально и этично.

Отчёт построен по трём этапам задания: разведка, обнаружение уязвимостей и их устранение. Для каждой уязвимости приведены способ обнаружения, практическая эксплуатация и развёрнутые рекомендации по исправлению с рекомендуемой моделью, кодом «было → стало» и поясняющей схемой «как неправильно / как надо». Надстрочные сноски дублируются внизу страницы и соответствуют нумерации в списке литературы.

---

[1] Электронный ресурс [https://owasp.org/www-project-juice-shop/] — OWASP Juice Shop Project, OWASP Foundation, дата обращения: 15.09.2026.
[2] Электронный ресурс [https://pwning.owasp-juice.shop/] — Pwning OWASP Juice Shop (companion guide), Björn Kimminich, дата обращения: 15.09.2026.
[3] Электронный ресурс [https://owasp.org/www-project-top-ten/] — OWASP Top 10, OWASP Foundation, дата обращения: 16.09.2026.

2. Этап разведки

Разведка (reconnaissance) проводилась до эксплуатации и охватывала всё приложение. Её цель — построить карту поверхности атаки: технологии, маршруты, API-эндпоинты, роли и точки входа.

Изучение клиентской части. Приложение — SPA на Angular; весь код поставляется в бандле main.js. Его анализ (поиск строк «admin», «administration», «/rest/», «/api/») восстановил список маршрутов и скрытых разделов, включая /#/administration и /#/score-board.

Анализ сетевого взаимодействия. Через DevTools и прокси Burp Suite фиксировались запросы к REST API: /rest/user/login, /rest/products/search, /rest/basket/{id}, /api/Users, /api/Feedbacks, /rest/products/reviews — с методами, параметрами и форматом ответа.

Перебор путей (directory busting). Обнаружены служебные ресурсы: каталог /ftp, эндпоинт /support/logs, резервные копии (package.json.bak).

Идентификация технологий и версий. По заголовкам, package.json и сообщениям об ошибках определены библиотеки и их версии, сопоставленные с известными CVE.

Снятие ролевой модели. Выявлены типовые учётные записи, механизм аутентификации на JWT в localStorage и места применения токена.

Итогом разведки стала карта поверхности атаки, на основе которой последовательно проверялись классы уязвимостей OWASP Top 10.

Краткое резюме обнаруженных уязвимостей

Ниже — краткие описания (что эксплуатирует, как активировать, как быстро исправить). Надстрочная сноска у каждой уязвимости ведёт к источнику внизу страницы и в списке литературы. Подробный разбор — в разделе 3.

№1. SQL-инъекция в форме входа (обход аутентификации) [A03:2021 – Injection]. Эксплуатирует конкатенацию пользовательского ввода в SQL-запрос на эндпоинте входа. Активируется payload'ом «' OR 1=1 --» в поле email. Исправляется параметризованными запросами (prepared statements).[4]

№2. UNION-based SQL-инъекция в поиске товаров (выгрузка БД) [A03:2021 – Injection]. Эксплуатирует подстановку параметра q в SQL-запрос поиска. Активируется UNION SELECT-инъекцией в строке поиска. Исправляется параметризацией и ORM.[5]

№3. NoSQL-инъекция (массовое изменение отзывов) [A03:2021 – Injection]. Эксплуатирует передачу MongoDB-операторов в теле запроса обновления отзыва. Активируется объектом {"$ne":-1} в поле id. Исправляется строгой валидацией типов.[6]

№4. DOM-based XSS в поле поиска [A03:2021 – Injection (XSS)]. Эксплуатирует вывод строки поиска в DOM без санитайзинга. Активируется payload'ом <iframe src="javascript:alert(`xss`)">. Исправляется экранированием и доверенным санитайзером.[7]

№5. Отражённый XSS в параметре отслеживания заказа [A03:2021 – Injection (XSS)]. Эксплуатирует отражение параметра id на странице трекинга заказа. Активируется XSS-payload'ом в query-параметре. Исправляется экранированием вывода.[8]

№6. Хранимый (persisted) XSS через HTTP-заголовок [A03:2021 – Injection (XSS)]. Эксплуатирует сохранение значения HTTP-заголовка

[4] Электронный ресурс [https://owasp.org/Top10/A03_2021-Injection/] — A03:2021 – Injection, OWASP Foundation, дата обращения: 15.09.2026.
[5] Электронный ресурс [https://portswigger.net/web-security/sql-injection/union-attacks] — SQL injection UNION attacks, PortSwigger Web Security Academy, дата обращения: 16.09.2026.
[6] Электронный ресурс [https://owasp.org/www-community/attacks/NoSQL_injection] — NoSQL injection, OWASP Community, дата обращения: 18.09.2026.
[7] Электронный ресурс [https://owasp.org/www-community/attacks/DOM_Based_XSS] — DOM Based XSS, OWASP Community, дата обращения: 19.09.2026.
[8] Электронный ресурс [https://owasp.org/www-community/attacks/xss/] — Cross Site Scripting (XSS), OWASP Community, дата обращения: 20.09.2026.

(например True-Client-IP) и его последующий вывод в админке. Активируется XSS в заголовке запроса. Исправляется экранированием при выводе.[9]

№7. Нарушение контроля доступа: доступ к разделу администрирования [A01:2021 – Broken Access Control]. Эксплуатирует отсутствие серверной проверки роли для админ-маршрута. Активируется прямым переходом на /#/administration. Исправляется серверной авторизацией по ролям.[10]

№8. IDOR: доступ к чужой корзине [A01:2021 – Broken Access Control]. Эксплуатирует отсутствие проверки владельца при обращении к корзине по id. Активируется подменой BasketId в URL/запросе. Исправляется проверкой принадлежности объекта пользователю.[11]

№9. Подделка отзыва от имени другого пользователя [A01:2021 – Broken Access Control]. Эксплуатирует доверие к полю author в теле запроса. Активируется подменой author при создании отзыва. Исправляется привязкой автора к серверной сессии.[12]

№10. Mass Assignment: регистрация с правами администратора [A01:2021 – Broken Access Control]. Эксплуатирует отсутствие белого списка полей при регистрации. Активируется добавлением "role":"admin" в тело запроса. Исправляется явным разрешённым набором полей.[13]

№11. Сброс пароля через слабый секретный вопрос [A07:2021 – Identification and Authentication Failures]. Эксплуатирует предсказуемые ответы на секретные вопросы восстановления. Активируется подбором ответа

[9] Электронный ресурс [https://portswigger.net/web-security/cross-site-scripting/stored] — Stored cross-site scripting, PortSwigger Web Security Academy, дата обращения: 22.09.2026.
[10] Электронный ресурс [https://owasp.org/Top10/A01_2021-Broken_Access_Control/] — A01:2021 – Broken Access Control, OWASP Foundation, дата обращения: 23.09.2026.
[11] Электронный ресурс [https://portswigger.net/web-security/access-control/idor] — Insecure direct object references (IDOR), PortSwigger Web Security Academy, дата обращения: 24.09.2026.
[12] Электронный ресурс [https://cwe.mitre.org/data/definitions/639.html] — CWE-639: Authorization Bypass Through User-Controlled Key, MITRE CWE, дата обращения: 25.09.2026.
[13] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html] — Mass Assignment Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 26.09.2026.

(OSINT) на /rest/user/reset-password. Исправляется отказом от секретных вопросов в пользу безопасного восстановления.[14]

№12. Отсутствие защиты от перебора (brute force) при входе [A07:2021 – Identification and Authentication Failures]. Эксплуатирует отсутствие rate limiting на входе. Активируется автоматическим перебором паролей (Hydra/Burp Intruder). Исправляется ограничением частоты и блокировками.[15]

№13. Раскрытие данных: доступ к каталогу /ftp [A05:2021 – Security Misconfiguration]. Эксплуатирует открытый статический каталог /ftp. Активируется прямым обращением к /ftp. Исправляется запретом листинга и выносом файлов за пределы веб-корня.[16]

№14. Обход фильтра расширений через Poison Null Byte (path traversal) [A05:2021 – Security Misconfiguration]. Эксплуатирует обрыв проверки расширения нулевым байтом. Активируется суффиксом %2500.md к имени файла в /ftp. Исправляется нормализацией пути и жёсткой проверкой.[17]

№15. Раскрытие конфиденциального документа о поглощениях [A01:2021 – Broken Access Control]. Эксплуатирует доступность внутреннего документа в /ftp. Активируется скачиванием acquisitions.md. Исправляется выносом документа из публичного каталога и контролем доступа.[18]

№16. Доступ к журналам сервера (/support/logs) [A09:2021 – Security Logging and Monitoring Failures]. Эксплуатирует открытый доступ к лог-

[14] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html] — Forgot Password Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 27.09.2026.
[15] Электронный ресурс [https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks] — Blocking Brute Force Attacks, OWASP Community, дата обращения: 28.09.2026.
[16] Электронный ресурс [https://owasp.org/Top10/A05_2021-Security_Misconfiguration/] — A05:2021 – Security Misconfiguration, OWASP Foundation, дата обращения: 29.09.2026.
[17] Электронный ресурс [https://owasp.org/www-community/attacks/Embedding_Null_Code] — Embedding Null Code (Poison Null Byte), OWASP Community, дата обращения: 30.09.2026.
[18] Электронный ресурс [https://cwe.mitre.org/data/definitions/200.html] — CWE-200: Exposure of Sensitive Information, MITRE CWE, дата обращения: 01.10.2026.

файлам. Активируется обращением к /support/logs. Исправляется ограничением доступа к журналам.[19]

№17. XXE: чтение файлов через загрузку XML [A05:2021 – Security Misconfiguration (XXE)]. Эксплуатирует обработку внешних сущностей XML-парсером. Активируется загрузкой XML с DOCTYPE/ENTITY file://. Исправляется отключением внешних сущностей (DTD).[20]

№18. SSRF через подмену URL профильного изображения [A10:2021 – Server-Side Request Forgery]. Эксплуатирует загрузку аватара по URL, задаваемому пользователем. Активируется указанием внутреннего URL в imageUrl. Исправляется белым списком адресов и запретом внутренних сетей.[21]

№19. Подделка JWT через алгоритм none / слабый секрет [A02:2021 – Cryptographic Failures]. Эксплуатирует приём токенов с alg:none или известным секретом. Активируется подменой payload и пересчётом/обнулением подписи. Исправляется жёсткой фиксацией алгоритма и секрета.[22]

№20. Слабое хеширование паролей (MD5) [A02:2021 – Cryptographic Failures]. Эксплуатирует хранение паролей в виде несолёного MD5. Активируется офлайн-перебором хешей после выгрузки БД. Исправляется переходом на bcrypt/argon2 с солью.[23]

№21. Небезопасная десериализация / выполнение кода [A08:2021 – Software and Data Integrity Failures]. Эксплуатирует небезопасную обработку пользовательских данных в уязвимой зависимости. Активируется специально

[19] Электронный ресурс [https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/] — A09:2021 – Security Logging and Monitoring Failures, OWASP Foundation, дата обращения: 02.10.2026.
[20] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html] — XML External Entity Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 03.10.2026.
[21] Электронный ресурс [https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/] — A10:2021 – Server-Side Request Forgery (SSRF), OWASP Foundation, дата обращения: 05.10.2026.
[22] Электронный ресурс [https://portswigger.net/web-security/jwt] — JWT attacks, PortSwigger Web Security Academy, дата обращения: 06.10.2026.
[23] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html] — Password Storage Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 07.10.2026.

сформированной нагрузкой. Исправляется обновлением зависимостей и валидацией входных данных.[24]

№22. Уязвимые и устаревшие компоненты (перезапись legal.md) [A06:2021 – Vulnerable and Outdated Components]. Эксплуатирует уязвимость устаревшей зависимости обработки архивов/путей. Активируется загрузкой архива с path traversal. Исправляется обновлением компонентов и проверкой путей.[25]

№23. Обход защиты от автоматизации (10+ отзывов за 10 секунд) [A04:2021 – Insecure Design]. Эксплуатирует отсутствие серверного ограничения частоты отправки форм. Активируется скриптовой отправкой множества запросов. Исправляется server-side rate limiting и CAPTCHA.[26]

№24. Отсутствие/слабость заголовков безопасности (CSP) [A05:2021 – Security Misconfiguration]. Эксплуатирует слабую политику Content-Security-Policy, допускающую инлайновые скрипты. Активируется XSS-нагрузкой, не блокируемой CSP. Исправляется строгой CSP и security-заголовками.[27]

№25. Слабая криптография купонов (предсказуемые скидки) [A02:2021 – Cryptographic Failures]. Эксплуатирует обратимое/слабое кодирование кодов купонов. Активируется генерацией валидного купона по вскрытому алгоритму. Исправляется криптостойкой генерацией и серверной проверкой.[28]

№26. Безопасность через сокрытие (утечка до анонса) [A04:2021 – Insecure Design]. Эксплуатирует доступность «скрытых» URL/ресурсов до

[24] Электронный ресурс [https://owasp.org/www-community/vulnerabilities/Deserialization_of_untrusted_data] — Deserialization of untrusted data, OWASP Community, дата обращения: 08.10.2026.
[25] Электронный ресурс [https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/] — A06:2021 – Vulnerable and Outdated Components, OWASP Foundation, дата обращения: 09.10.2026.
[26] Электронный ресурс [https://owasp.org/www-project-automated-threats-to-web-applications/] — OWASP Automated Threats to Web Applications, OWASP Foundation, дата обращения: 10.10.2026.
[27] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html] — Content Security Policy Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 11.10.2026.
[28] Электронный ресурс [https://owasp.org/Top10/A02_2021-Cryptographic_Failures/] — A02:2021 – Cryptographic Failures, OWASP Foundation, дата обращения: 12.10.2026.

официального анонса. Активируется прямым обращением к неанонсированному пути. Исправляется реальным контролем доступа, а не сокрытием.[29]

№27. Небезопасная загрузка файлов (тип и размер) [A04:2021 – Insecure Design]. Эксплуатирует слабую проверку типа/размера загружаемого файла. Активируется загрузкой файла запрещённого типа или превышающего лимит. Исправляется строгой серверной валидацией.[30]

3. Обнаруженные уязвимости

Каждая уязвимость разобрана по трём пунктам: как нашли, как использовать, как исправить. Эксплуатация описана для стенда «Windows 10 (Juice Shop) + Kali Linux».

Уязвимость №1. SQL-инъекция в форме входа (обход аутентификации)

Категория OWASP: A03:2021 – Injection. **Критичность:** Критическая.

3.1.1. Как нашли

На этапе разведки в DevTools (вкладка Network браузера) было замечено, что форма входа отправляет POST-запрос на эндпоинт /rest/user/login с JSON-телом {"email":"...","password":"..."}. При вводе в поле email одинарной кавычки (') сервер вернул ответ 500 Internal Server Error, а в теле ответа раскрылся фрагмент SQL-запроса и трассировка стека Sequelize (SQLITE_ERROR: unrecognized token). Это прямой индикатор того, что ввод подставляется в SQL-запрос без экранирования. В исходном коде уязвимость находится в файле routes/login.ts (около строки 34), где email конкатенируется в строку запроса через шаблонный литерал: `SELECT * FROM Users WHERE email = '${req.body.email}' AND password = '...'`.

[29] Электронный ресурс [https://cwe.mitre.org/data/definitions/656.html] — CWE-656: Reliance on Security Through Obscurity, MITRE CWE, дата обращения: 13.10.2026.
[30] Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html] — File Upload Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 14.10.2026.

3.1.2. Как использовать

В поле email вводится payload: ' OR 1=1 -- (с пробелом после двойного дефиса), поле пароля заполняется произвольным значением. Комментарий -- отсекает проверку пароля, а условие OR 1=1 делает выборку истинной для всех строк; СУБД возвращает первую запись — администратора (admin@juice-sh.op), и сервер выдаёт валидный JWT. Для входа под конкретным пользователем применяется payload вида admin@juice-sh.op'-- . В Kali Linux это удобно воспроизводится через curl:

```
curl -s http://<IP_Windows>:3000/rest/user/login -H 'Content-Type:
application/json' -d '{"email":"\' OR 1=1 --","password":"x"}'
```

В ответе приходит authentication.token — JWT администратора.

3.1.3. Как исправить

Рекомендуемая модель: Параметризация запросов + ORM + принцип наименьших привилегий.

Коренная причина — смешивание кода (текста SQL) и данных (ввода пользователя) в одной строке. Правильная модель полностью разделяет их: текст запроса фиксирован, а пользовательские значения передаются отдельным каналом как параметры-плейсхолдеры, которые драйвер БД экранирует и никогда не интерпретирует как SQL. В стеке Juice Shop (Sequelize) для этого есть штатные методы модели (findOne/findAll с объектом where) и, при необходимости «сырого» SQL, опция replacements/bind. Дополнительно: валидация формата email на входе (express-validator), выдача одинаковой ошибки при неверном логине и пароле (чтобы не раскрывать, что именно неверно), и подключение к БД под учётной записью с минимально необходимыми правами (без DROP/ALTER).

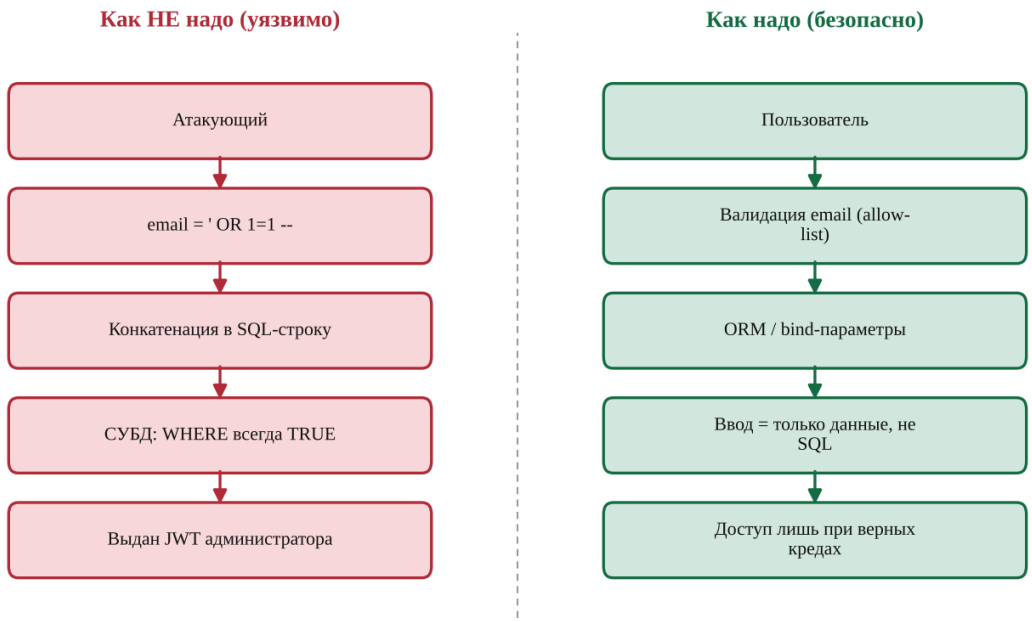


Рисунок 1 — Уязвимый и безопасный поток обработки (SQL-инъекция в форме входа (обход аутентификации))

Было (уязвимо):

```
// routes/login.ts — УЯЗВИМО: ввод конкатенируется в SQL
models.sequelize.query(
  `SELECT * FROM Users
   WHERE email = '${req.body.email}'
     AND password = '${security.hash(req.body.password)}'
     AND deletedAt IS NULL`,
  { model: models.User, plain: true }
).then(authenticatedUser => { /* выдать JWT */ })
```

Стало (безопасно):

```
// routes/login.ts — БЕЗОПАСНО: параметризация + ORM
import { body, validationResult } from 'express-validator'

export const login = [
  body('email').isEmail().normalizeEmail(),
  body('password').isString().isLength({ min: 1 }),
  async (req, res) => {
    if (!validationResult(req).isEmpty())
      return res.status(400).json({ error: 'Invalid input' })

    // Вариант 1 — методы ORM (значения параметризуются автоматически)
    const user = await models.User.findOne({
      where: { email: req.body.email, deletedAt: null }
    })
    // пароль сверяем отдельно безопасным сравнением хешей
```

```
    if (!user || !(await bcrypt.compare(req.body.password,
user.password)))
      return res.status(401).json({ error: 'Invalid credentials' })

    // Вариант 2 — если нужен сырой SQL, только через bind-параметры:
    // models.sequelize.query(
    //   'SELECT * FROM Users WHERE email = $email AND deletedAt IS NULL',
    //   { bind: { email: req.body.email }, model: models.User })
    return res.json({ token: security.authorize(user) })
  }
]
```

Уязвимость №2. UNION-based SQL-инъекция в поиске товаров (выгрузка БД)

Категория OWASP: A03:2021 – Injection. **Критичность:** Критическая.

3.2.1. Как нашли

В разведке установлено, что строка поиска обращается к GET /rest/products/search?q=<строка>. При добавлении кавычки q=test' сервер возвращает 500 и SQL-ошибку, что подтверждает инъекцию. Уязвимость — в routes/search.ts, где параметр q подставляется напрямую в SQL без параметризации.

3.2.2. Как использовать

Сначала определяется число столбцов исходного SELECT (через ORDER BY либо подбор NULL в UNION). Затем конструируется payload, выгружающий данные из таблицы Users:

```
/rest/products/search?q=qwert'))  UNION SELECT id, email, password,
'4','5','6','7','8','9' FROM Users--
```

В ответе в полях товаров отображаются email и MD5-хеши паролей всех пользователей. Для выгрузки схемы БД используется запрос к sqlite_master (challenge «Database Schema»): ... UNION SELECT sql,2,3,4,5,6,7,8,9 FROM sqlite_master--.

3.2.3. Как исправить

Рекомендуемая модель: Параметризованный поиск через ORM (Op.like) + ограничение прав БД.

Поиск строится тем же небезопасным способом — подстановкой q в текст SQL, что позволяет дописать UNION SELECT и прочитать произвольные таблицы. Модель исправления: поиск по подстроке выполнять оператором LIKE с параметром-плейсхолдером через ORM (Sequelize Op.like), никогда не собирая SQL вручную. Возвращать клиенту только нужные поля (явный attributes), чтобы даже при ошибке не утекали служебные столбцы. На уровне БД — отдельный пользователь только с SELECT на таблицу товаров, без доступа к Users и sqlite_master.

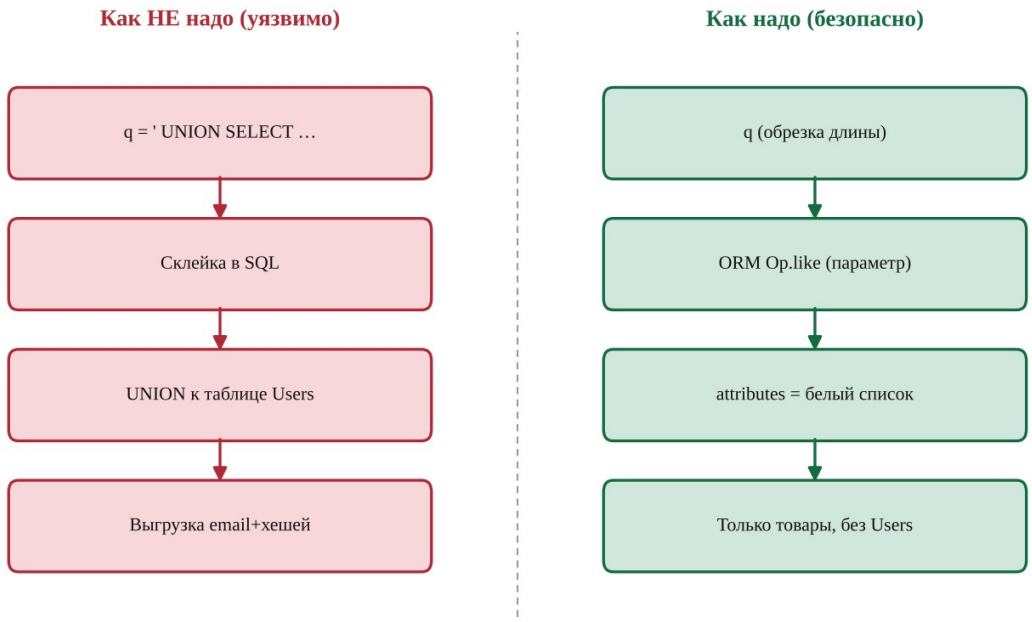


Рисунок 2 — Уязвимый и безопасный поток обработки (UNION-based SQL-инъекция в поиске товаров (выгрузка БД))

Было (уязвимо):

```
// routes/search.ts — УЯЗВИМО
models.sequelize.query(
  `SELECT * FROM Products
   WHERE ((name LIKE '%${req.query.q}%'
```

```
       OR description LIKE '%${req.query.q}%')
      AND deletedAt IS NULL) ORDER BY name`
)
```

Стало (безопасно):

```
// routes/search.ts — БЕЗОПАСНО
import { Op } from 'sequelize'

const q = String(req.query.q ?? '').slice(0, 100)  // нормализация
const products = await models.Product.findAll({
  attributes: ['id', 'name', 'description', 'price', 'image'], // белый
список полей
  where: {
    deletedAt: null,
    [Op.or]: [
      { name:        { [Op.like]: `%${q}%` } },  // q уходит как параметр
      { description: { [Op.like]: `%${q}%` } }
    ]
  },
  order: [['name', 'ASC']]
})
```

Уязвимость №3. NoSQL-инъекция (массовое изменение отзывов)

Категория OWASP: A03:2021 – Injection. **Критичность:** Высокая.

3.3.1. Как нашли

Отзывы к товарам хранятся в MongoDB-совместимом хранилище (MarsDB). При анализе запроса PATCH /rest/products/reviews выяснилось, что поле id из тела запроса попадает в селектор запроса без проверки типа, что типично для NoSQL-инъекций.

3.3.2. Как использовать

Вместо конкретного строкового id в тело запроса передаётся объект-оператор MongoDB, истинный для всех документов:

```
PATCH /rest/products/reviews  { "id": { "$ne": -1 }, "message": "NoSQL
pwned" }
```

Оператор $ne (не равно) совпадает со всеми отзывами, и текст заменяется сразу во всех документах. Аналогично оператором $gt/$where можно устроить отказ в обслуживании (NoSQL DoS).

3.3.3. Как исправить

Рекомендуемая модель: Строгая типизация ввода + санитайзинг операторов ($).

NoSQL-инъекция возникает, когда значение-объект пользователя попадает в селектор запроса и интерпретируется как оператор ($ne, $gt, $where). Модель: приводить идентификатор к примитивному типу (строке) и отклонять любой объект; глобально удалять из тела запроса ключи, начинающиеся с '$' или содержащие '.', промежуточным middleware (express-mongo-sanitize). Операцию обновления ограничивать конкретным документом, а не множеством.

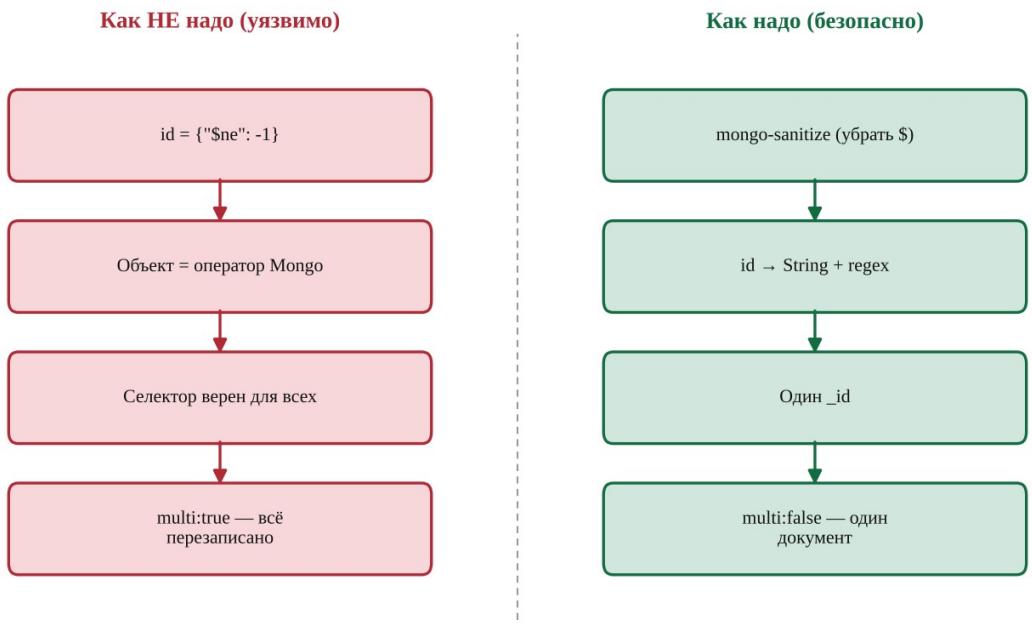


Рисунок 3 — Уязвимый и безопасный поток обработки (NoSQL-инъекция (массовое изменение отзывов))

Было (уязвимо):

```
// УЯЗВИМО: id-объект попадает в селектор как оператор
app.patch('/rest/products/reviews', (req, res) => {
  db.reviews.update(
    { _id: req.body.id },          // { $ne: -1 } → совпадает со всеми
    { $set: { message: req.body.message } },
    { multi: true }                // массовое изменение
  )
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: типизация + запрет операторов + один документ
const mongoSanitize = require('express-mongo-sanitize')
app.use(mongoSanitize())          // вырезает ключи с $ и .

app.patch('/rest/products/reviews', (req, res) => {
  const id = String(req.body.id)   // только строка
  if (!/^[a-f0-9]{24}$/.test(id))  // ожидаемый формат ObjectId
    return res.status(400).json({ error: 'Invalid id' })
  db.reviews.update(
    { _id: id },
    { $set: { message: String(req.body.message).slice(0, 2000) } },
    { multi: false }               // ровно один документ
  )
})
```

Уязвимость №4. DOM-based XSS в поле поиска

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.4.1. Как нашли

Значение параметра поиска отражается обратно на странице (заголовок «Search Results - ...»). Angular по умолчанию экранирует вывод, но в коде используется обход через bypassSecurityTrustHtml/innerHTML, что открывает DOM XSS.

3.4.2. Как использовать

В строку поиска вводится полезная нагрузка:

```
<iframe src="javascript:alert(`xss`)">
```

После отправки скрипт выполняется в контексте страницы (срабатывает alert). Нагрузка также передаётся через URL #/search?q=<iframe...>, что позволяет доставлять её жертве ссылкой.

3.4.3. Как исправить

Рекомендуемая модель: Отказ от bypassSecurityTrustHtml + санитайзер (DOMPurify) + CSP.

DOM XSS появляется, когда недоверенная строка попадает в HTML-контекст (innerHTML) в обход авто-экранирования Angular. Модель: по умолчанию выводить данные через интерполяцию {{ }} и property binding, которые Angular экранирует сам; если HTML действительно нужен — прогонять его через DOMPurify и только потом доверять. Никогда не использовать bypassSecurityTrustHtml для пользовательского ввода. Усилить защиту строгой CSP, запрещающей инлайновые скрипты.

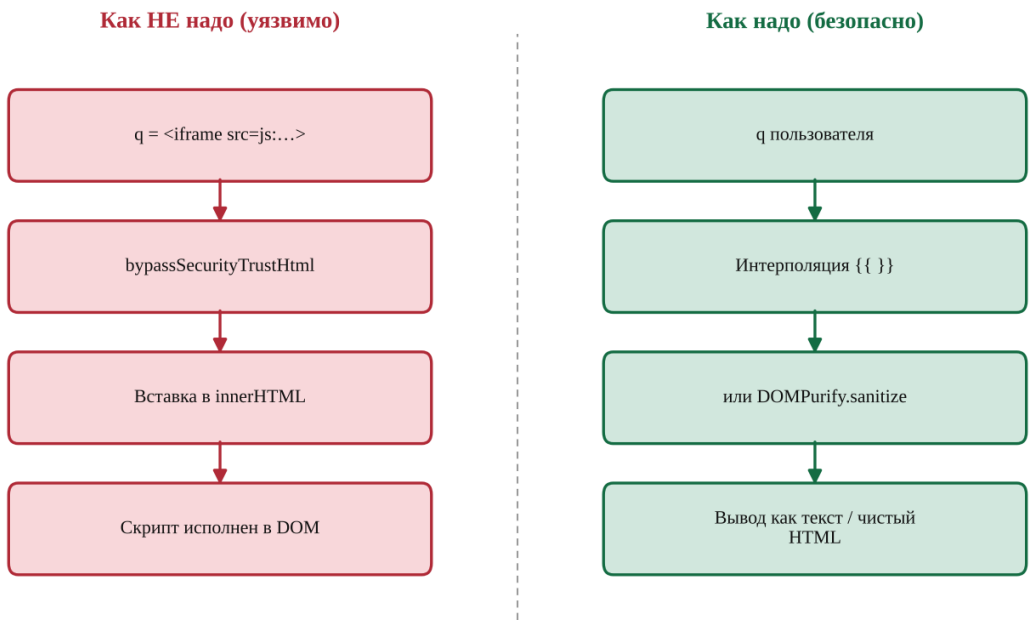


Рисунок 4 — Уязвимый и безопасный поток обработки (DOM-based XSS в поле поиска)

Было (уязвимо):

```
// Angular-компонент — УЯЗВИМО
this.searchValue = this.sanitizer.bypassSecurityTrustHtml(queryParam)
// template: <span [innerHTML]="searchValue"></span>  → скрипт исполняется
```

Стало (безопасно):

```
// БЕЗОПАСНО
// template: <span>{{ searchValue }}</span>   // Angular экранирует сам
this.searchValue = queryParam

// Если HTML всё же необходим — очищаем DOMPurify:
import DOMPurify from 'dompurify'
this.safeHtml = DOMPurify.sanitize(rawHtml, { USE_PROFILES: { html: true }
})
```

```
// template: <div [innerHTML]="safeHtml"></div>
```

Уязвимость №5. Отражённый XSS в параметре отслеживания заказа

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.5.1. Как нашли

Страница отслеживания заказа (/#/track-result?id=<значение>) отражает параметр id без обработки. При подстановке HTML-тега он интерпретируется браузером, что указывает на отражённый XSS.

3.5.2. Как использовать

Формируется ссылка с нагрузкой в параметре id:

```
http://<IP>:3000/#/track-result?id=<iframe
src="javascript:alert(`xss`)">
```

При открытии жертвой скрипт выполняется в её сессии. Такой вектор используется для фишинга и кражи токена из localStorage.

3.5.3. Как исправить

Рекомендуемая модель: Контекстное экранирование вывода + валидация параметра + CSP.

Отражённый XSS — частный случай: параметр из URL сразу попадает в страницу без экранирования. Модель та же, что для DOM XSS: выводить через безопасные механизмы фреймворка, валидировать ожидаемый формат параметра (id заказа — это шаблон, а не произвольный HTML) и включить CSP. На сервере при формировании любого HTML применять экранирование (например he.encode).

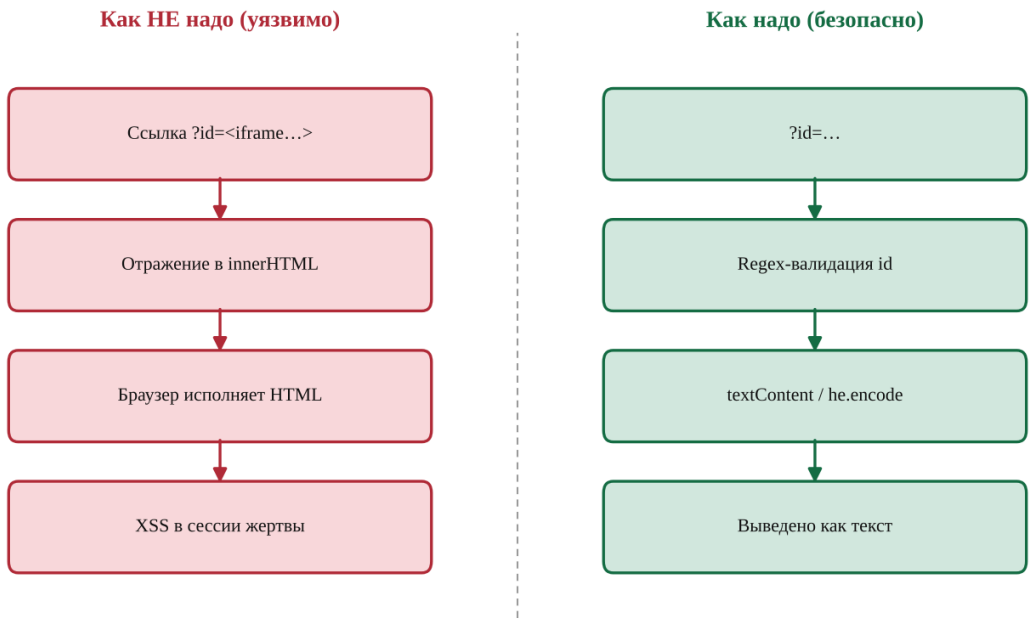


Рисунок 5 — Уязвимый и безопасный поток обработки (Отражённый XSS в параметре отслеживания заказа)

Было (уязвимо):

```
// УЯЗВИМО: id из URL отражается как HTML
element.innerHTML = 'Заказ: ' + params.id   // <iframe …> исполнится
```

Стало (безопасно):

```
// БЕЗОПАСНО: валидация формата + текстовый вывод
const id = params.id
if (!/^[0-9a-f-]{6,40}$/i.test(id))
  throw new Error('Invalid order id')
element.textContent = 'Заказ: ' + id   // как текст, не HTML
// серверный рендер: res.send(`Заказ: ${he.encode(id)}`)
```

Уязвимость №6. Хранимый (persisted) XSS через HTTP-заголовок

Категория OWASP: A03:2021 – Injection (XSS). **Критичность:** Высокая.

3.6.1. Как нашли

Часть служебных данных (IP-адрес последнего входа, User-Agent) сохраняется и позднее отображается в интерфейсе. Значение берётся из

управляемого клиентом HTTP-заголовка и выводится без экранирования — классический хранимый XSS.

3.6.2. Как использовать

В Kali через Burp Suite или curl в запрос добавляется заголовок с нагрузкой, например:

```
True-Client-IP: <iframe src="javascript:alert(`xss`)">
```

Значение сохраняется на сервере и срабатывает, когда администратор просматривает соответствующую страницу (хранимый характер делает атаку особенно опасной — она поражает других пользователей).

3.6.3. Как исправить

Рекомендуемая модель: Экранирование при выводе из любого источника + недоверие к заголовкам.

Хранимый XSS опаснее всего: нагрузка сохраняется и бьёт по другим пользователям (например администратору). Корневая ошибка — доверие к данным из HTTP-заголовков и вывод их без экранирования. Модель: любые данные при выводе в HTML экранировать независимо от источника; заголовки, управляемые клиентом, нормализовать/валидировать при сохранении; токен хранить в HttpOnly-куке, чтобы XSS не мог его украсть; включить CSP.

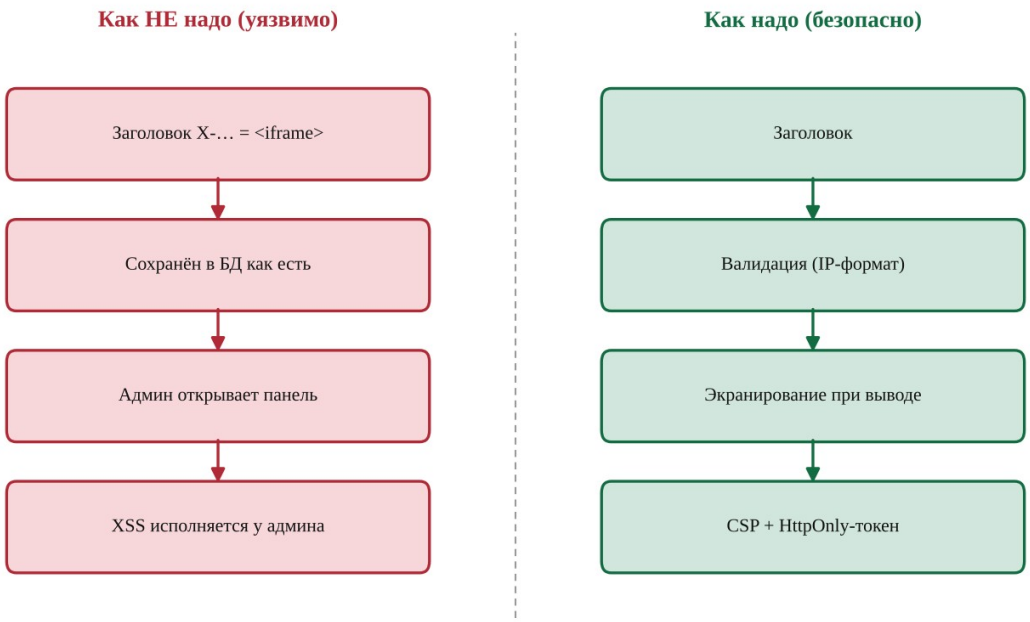


Рисунок 6 — Уязвимый и безопасный поток обработки (Хранимый (persisted) XSS через HTTP-заголовок)

Было (уязвимо):

```
// УЯЗВИМО: заголовок сохраняется и выводится как HTML
user.lastLoginIp = req.headers['true-client-ip']   // любой текст
// admin-панель: <td [innerHTML]="user.lastLoginIp"></td>
```

Стало (безопасно):

```
// БЕЗОПАСНО: валидация при сохранении + экранирование при выводе
import ipaddr from 'ipaddr.js'
const raw = String(req.headers['true-client-ip'] ?? '')
user.lastLoginIp = ipaddr.isValid(raw) ? raw : 'unknown' // только
валидный IP

// template (Angular экранирует): <td>{{ user.lastLoginIp }}</td>
// токен: res.cookie('token', jwt, { httpOnly: true, secure: true,
sameSite: 'strict' })
```

Уязвимость №7. Нарушение контроля доступа: доступ к разделу администрирования

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.7.1. Как нашли

В собранном JavaScript-бандле (main.js) при поиске строки «administration» обнаружен клиентский маршрут /#/administration. Пункт меню скрыт в UI, но маршрут не защищён на сервере — доступ ограничен лишь на уровне фронтенда.

3.7.2. Как использовать

В адресной строке напрямую открывается http://<IP>:3000/#/administration. Страница администрирования отображается обычному аутентифицированному пользователю, раскрывая список заказов и отзывов. Это демонстрирует, что контроль доступа реализован только на клиенте.

3.7.3. Как исправить

Рекомендуемая модель: Серверная авторизация по ролям (RBAC), запрет по умолчанию.

Скрытие пункта меню в UI — не защита: маршрут и его API доступны напрямую. Модель: каждый защищённый эндпоинт проверяет роль на сервере через middleware; доступ запрещён по умолчанию и выдаётся явно. Клиентские guard'ы (Angular CanActivate) — лишь удобство UX, но не граница безопасности.

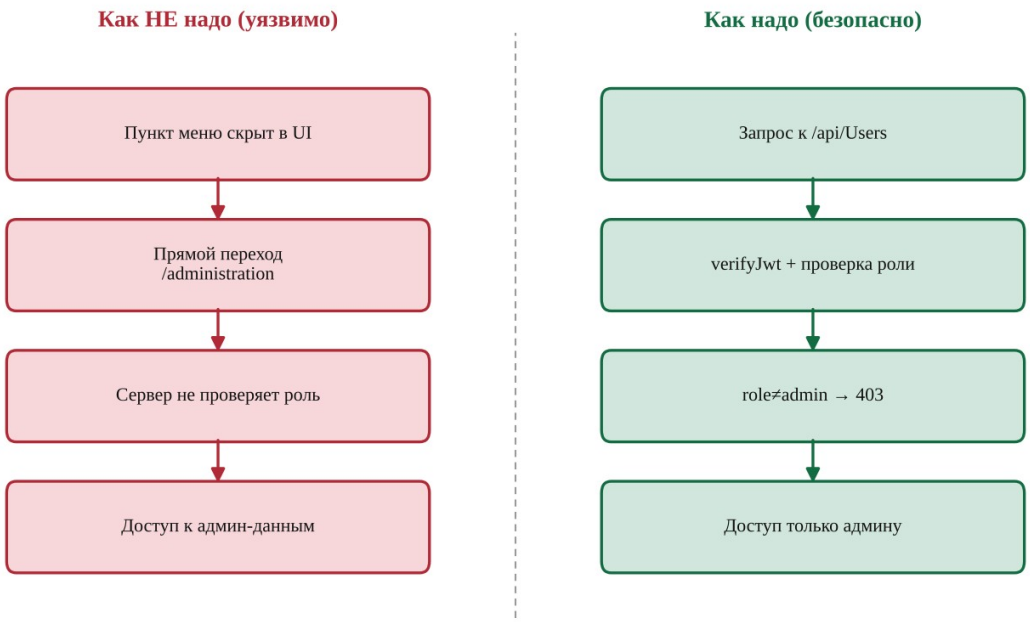


Рисунок 7 — Уязвимый и безопасный поток обработки (Нарушение контроля доступа: доступ к разделу администрирования)

Было (уязвимо):

```
// УЯЗВИМО: защита только на клиенте
// app-routing: { path: 'administration', component: AdminComponent }
// сервер отдаёт /api/Users, /api/Feedbacks без проверки роли
app.get('/api/Users', (req, res) => res.json(getAllUsers()))
```

Стало (безопасно):

```
// БЕЗОПАСНО: серверный RBAC-middleware
function requireRole(role) {
  return (req, res, next) => {
    const user = verifyJwt(req.headers.authorization) // проверка подписи
    if (!user || user.role !== role)
      return res.status(403).json({ error: 'Forbidden' })
    next()
  }
}
app.get('/api/Users', requireRole('admin'), (req, res) =>
res.json(getAllUsers()))
// клиентский CanActivate — только для UX, не замена серверной проверки
```

Уязвимость №8. IDOR: доступ к чужой корзине

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.8.1. Как нашли

Запрос GET /rest/basket/<id> возвращает содержимое корзины по числовому идентификатору. При изменении id на соседнее значение сервер отдаёт чужую корзину — признак небезопасной прямой ссылки на объект (IDOR).

3.8.2. Как использовать

После входа перехватывается запрос к своей корзине (например /rest/basket/1) и id меняется на 2, 3 и т.д. Сервер возвращает корзины других пользователей без проверки, что текущий пользователь — их владелец.

3.8.3. Как исправить

Рекомендуемая модель: Проверка владения объектом (ownership check) по идентификатору из токена.

IDOR — доступ к чужому объекту по предсказуемому id без проверки владельца. Модель: идентификатор пользователя брать ТОЛЬКО из проверенного JWT, а не из запроса, и сверять, что запрашиваемый объект принадлежит ему. Непредсказуемые идентификаторы (UUID) — дополнительный, но не основной барьер.

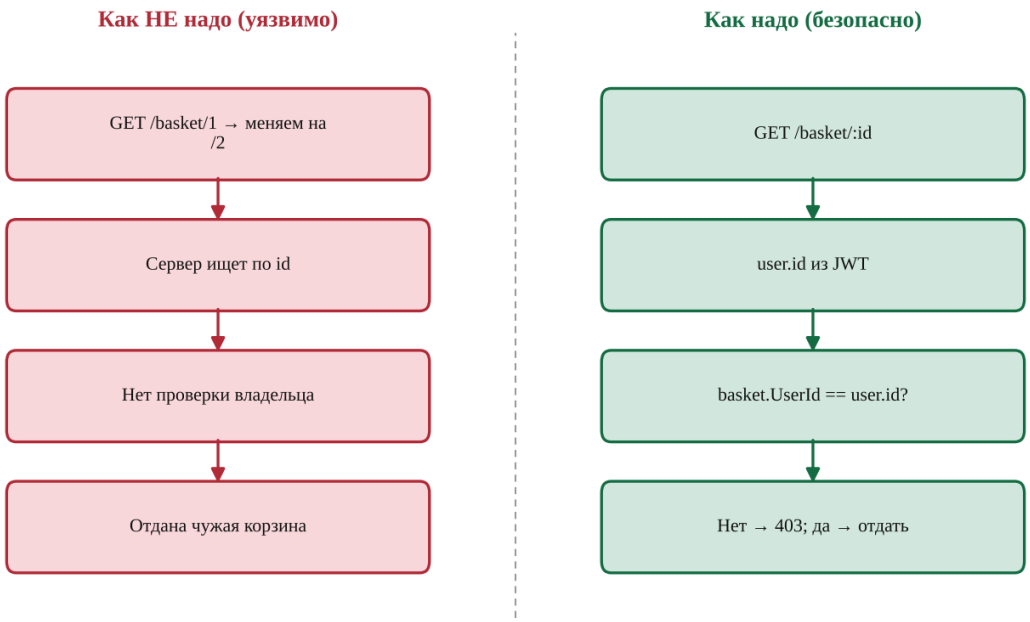


Рисунок 8 — Уязвимый и безопасный поток обработки (IDOR: доступ к чужой корзине)

Было (уязвимо):

```
// УЯЗВИМО: корзина отдаётся по id без проверки владельца
app.get('/rest/basket/:id', (req, res) => {
  const basket = models.Basket.findByPk(req.params.id)
  res.json(basket)            // чужой id → чужая корзина
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: сверка владельца
app.get('/rest/basket/:id', async (req, res) => {
  const user = verifyJwt(req.headers.authorization)
  const basket = await models.Basket.findByPk(req.params.id)
  if (!basket || basket.UserId !== user.id)
    return res.status(403).json({ error: 'Forbidden' })
  res.json(basket)
})
```

Уязвимость №9. Подделка отзыва от имени другого пользователя

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Средняя.

3.9.1. Как нашли

Создание отзыва (PUT /rest/products/<id>/reviews) принимает поле author из тела запроса и использует его как идентификатор автора, вместо того чтобы брать его из токена аутентификации.

3.9.2. Как использовать

В теле запроса поле author подменяется на чужой email (например admin@juice-sh.op). Отзыв сохраняется от имени указанного пользователя, что позволяет писать сообщения от чужого лица.

3.9.3. Как исправить

Рекомендуемая модель: Автор — всегда из серверной сессии, а не из тела запроса.

Уязвимость — доверие к полю author в запросе. Модель: игнорировать любые идентификаторы автора из тела и подставлять личность из проверенного токена. Это закрывает подделку сообщений от чужого лица.

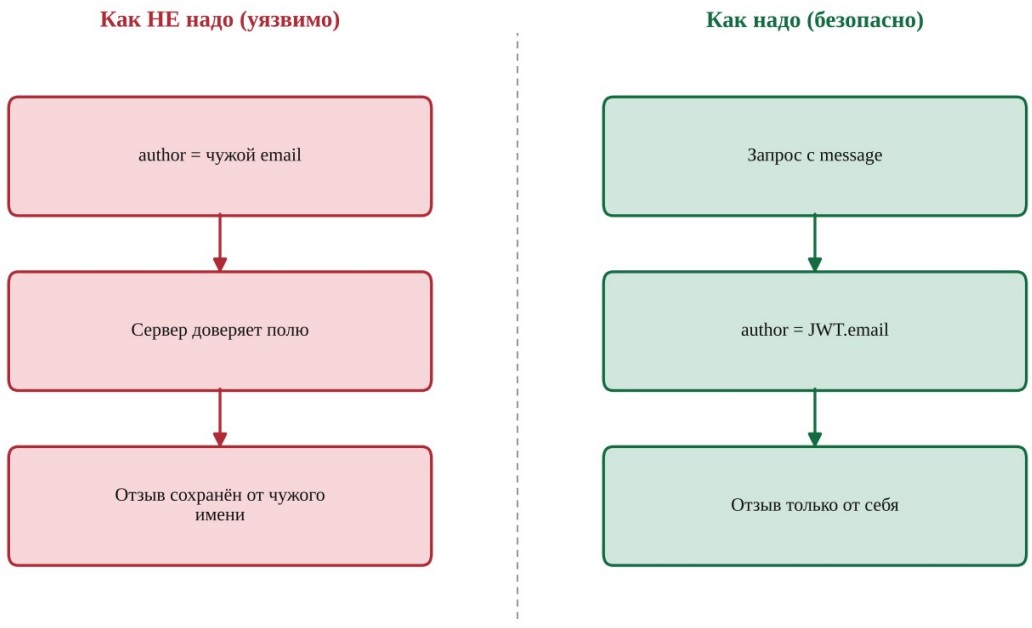


Рисунок 9 — Уязвимый и безопасный поток обработки (Подделка отзыва от имени другого пользователя)

Было (уязвимо):

```
// УЯЗВИМО: author берётся из запроса
db.reviews.insert({
  product: id,
  message: req.body.message,
  author: req.body.author      // можно подставить чужой email
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: author из токена
const user = verifyJwt(req.headers.authorization)
if (!user) return res.status(401).end()
db.reviews.insert({
  product: id,
  message: String(req.body.message).slice(0, 2000),
  author: user.email           // только собственная личность
})
```

Уязвимость №10. Mass Assignment: регистрация с правами администратора

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Высокая.

3.10.1. Как нашли

Эндпоинт регистрации POST /api/Users принимает JSON-объект и сохраняет его поля в модель пользователя. Поле role не входит в видимую форму, но модель его поддерживает — это уязвимость массового присваивания (mass assignment).

3.10.2. Как использовать

В тело запроса регистрации добавляется лишнее поле:

```
POST /api/Users  { "email":"evil@x.op", "password":"123",
"role":"admin" }
```

Созданный аккаунт получает роль admin, минуя обычную проверку привилегий.

3.10.3. Как исправить

Рекомендуемая модель: Белый список полей (allow-list) при привязке модели.

Mass assignment: клиент передаёт лишние поля (role), которые попадают в модель. Модель исправления — явно перечислять разрешённые к присвоению поля и отбрасывать всё остальное; роль назначать только серверной логикой. В Sequelize используется опция fields у create.

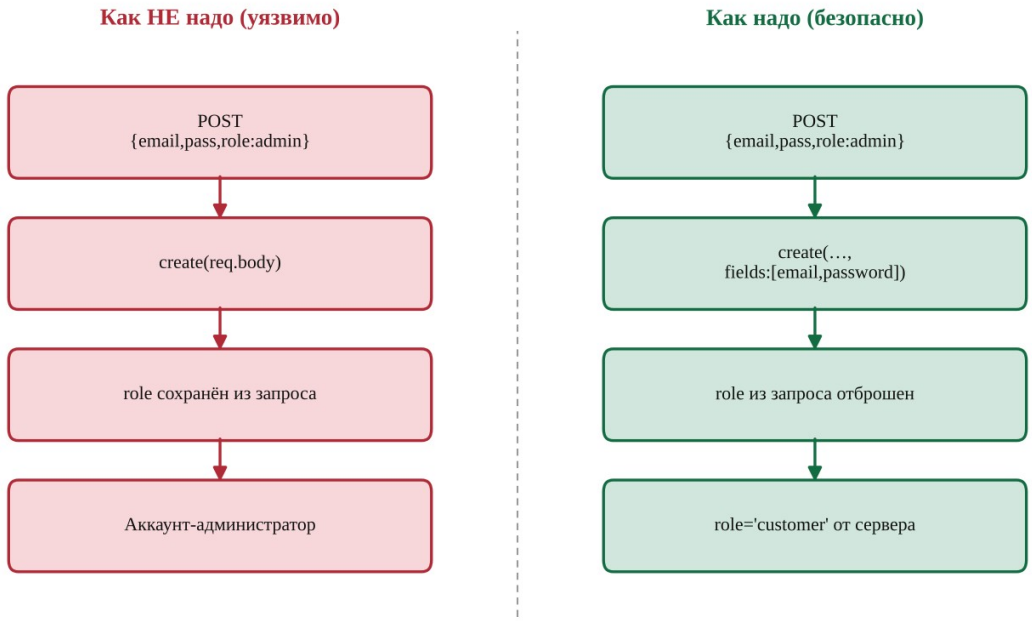


Рисунок 10 — Уязвимый и безопасный поток обработки (Mass Assignment: регистрация с правами администратора)

Было (уязвимо):

```
// УЯЗВИМО: вся модель заполняется из тела
app.post('/api/Users', (req, res) => {
  const user = models.User.create(req.body) // role:'admin' пройдёт
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: только разрешённые поля
app.post('/api/Users', async (req, res) => {
  const user = await models.User.create(req.body, {
    fields: ['email', 'password']   // allow-list; role игнорируется
```

```
  })
  user.role = 'customer'            // роль задаёт сервер
  await user.save()
})
```

Уязвимость №11. Сброс пароля через слабый секретный вопрос

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Высокая.

3.11.1. Как нашли

Механизм «Забыли пароль» использует секретный вопрос (например «Ваш любимый питомец?»). Ответы часто публичны или легко угадываются, а эндпоинт /rest/user/reset-password не ограничивает число попыток.

3.11.2. Как использовать

Для целевого аккаунта (например Jim — отсылка к Star Trek, ответ «Samuel») подбирается ответ на секретный вопрос; затем через /rest/user/reset-password задаётся новый пароль без знания старого. Данные для ответа добываются методами OSINT.

3.11.3. Как исправить

Рекомендуемая модель: Сброс по одноразовой ссылке на e-mail + rate-limit (отказ от секретных вопросов).

Секретные вопросы — слабый фактор: ответы публичны/угадываемы, а попытки не ограничены. Модель: сброс через одноразовый криптослучайный токен, отправляемый на подтверждённый email, с коротким сроком жизни и одноразовым использованием; ограничение числа попыток; уведомление пользователя о смене пароля.

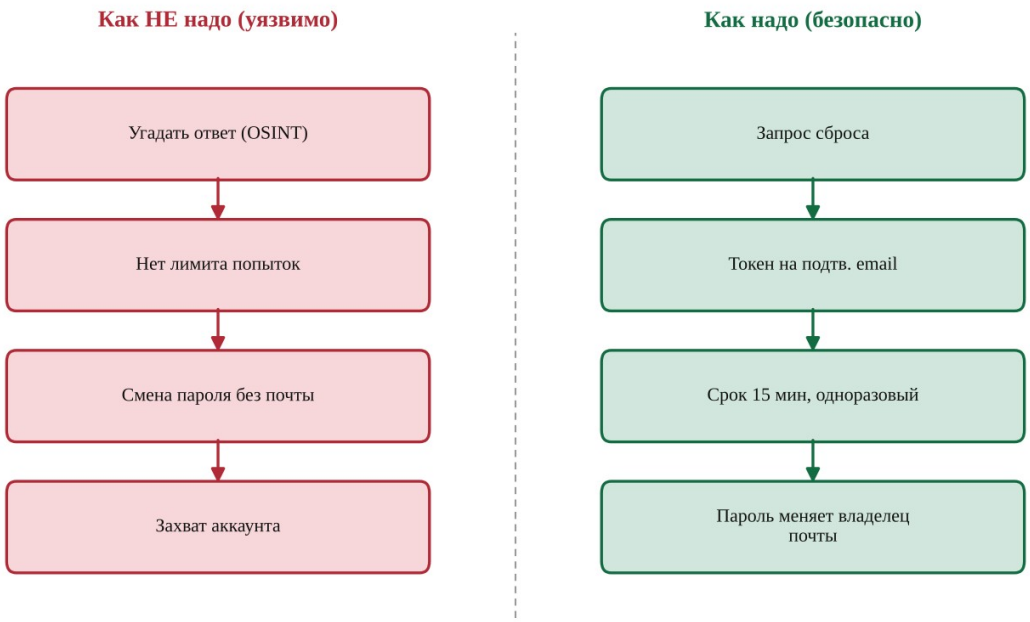


Рисунок 11 — Уязвимый и безопасный поток обработки (Сброс пароля через слабый секретный вопрос)

Было (уязвимо):

```
// УЯЗВИМО: смена пароля по ответу на секретный вопрос, без лимита
app.post('/rest/user/reset-password', (req, res) => {
  if (user.securityAnswer === req.body.answer)  // угадываемо
    user.setPassword(req.body.new)              // без ограничений
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: одноразовый токен на email + лимит
import crypto from 'crypto'
// шаг 1: запрос сброса
const token = crypto.randomBytes(32).toString('hex')
await db.resetTokens.insert({ userId: user.id, token,
  expiresAt: Date.now() + 15*60*1000, used: false })
sendMail(user.email, `/reset?token=${token}`)   // только владельцу почты

// шаг 2: установка пароля
const t = await db.resetTokens.findOne({ token: req.body.token })
if (!t || t.used || t.expiresAt < Date.now())
  return res.status(400).json({ error: 'Invalid or expired token' })
await setStrongPassword(t.userId, req.body.new)
await db.resetTokens.update({ token }, { used: true })
```

Уязвимость №12. Отсутствие защиты от перебора (brute force) при входе

Категория OWASP: A07:2021 – Identification and Authentication Failures.

Критичность: Средняя.

3.12.1. Как нашли

Эндпоинт /rest/user/login не ограничивает число попыток и не применяет CAPTCHA. Это позволяет автоматизировать перебор учётных данных.

3.12.2. Как использовать

В Kali Linux по словарю запускается перебор, например:

```
hydra -l admin@juice-sh.op -P rockyou.txt <IP> http-post-form ...
```

либо через Burp Intruder по эндпоинту /rest/user/login. Слабые пароли известных аккаунтов быстро подбираются.

3.12.3. Как исправить

Рекомендуемая модель: Rate-limiting + временная блокировка + CAPTCHA + MFA.

Отсутствие ограничений позволяет перебор. Модель: ограничивать частоту запросов к /login по IP и по аккаунту (express-rate-limit), вводить экспоненциальную задержку/временную блокировку после серии неудач, CAPTCHA при подозрении и многофакторную аутентификацию; требовать стойкие пароли и проверять их по базам утечек.

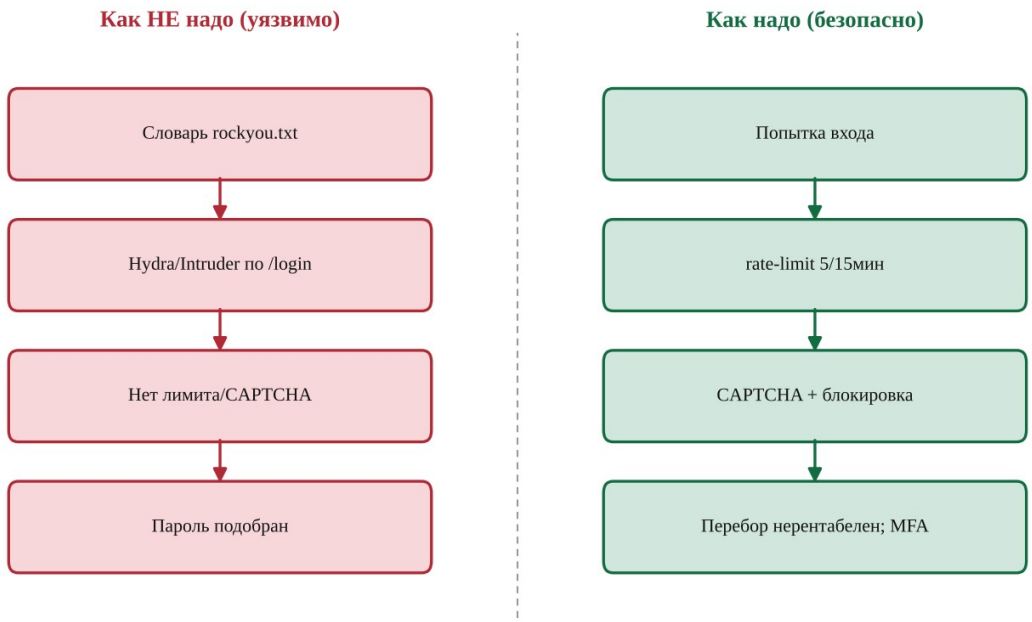


Рисунок 12 — Уязвимый и безопасный поток обработки (Отсутствие защиты от перебора (brute force) при входе)

Было (уязвимо):

```
// УЯЗВИМО: без ограничений
app.post('/rest/user/login', loginHandler)   // сколько угодно попыток
```

Стало (безопасно):

```
// БЕЗОПАСНО: rate-limit + блокировка
import rateLimit from 'express-rate-limit'
const loginLimiter = rateLimit({
  windowMs: 15*60*1000,       // окно 15 минут
  max: 5,                     // не более 5 попыток
  standardHeaders: true,
  message: { error: 'Too many attempts, try later' }
})
app.post('/rest/user/login', loginLimiter, loginHandler)
// + CAPTCHA после 3 неудач, + TOTP (MFA)
```

Уязвимость №13. Раскрытие данных: доступ к каталогу /ftp

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Средняя.

3.13.1. Как нашли

При переборе путей (directory busting) обнаружен доступный каталог http://<IP>:3000/ftp с листингом файлов, часть которых — служебные и конфиденциальные документы.

3.13.2. Как использовать

Открывается http://<IP>:3000/ftp — отображается список файлов (quarantine, acquisitions.md, legal.md, package.json.bak и др.), которые можно скачать напрямую.

3.13.3. Как исправить

Рекомендуемая модель: Запрет листинга каталогов + вынос файлов из веб-корня + контроль доступа.

Открытый статический каталог раскрывает файлы. Модель: не раздавать служебные директории статикой, отключить автоиндекс, конфиденциальные файлы хранить вне веб-корня и отдавать только авторизованным пользователям через контролируемый обработчик.

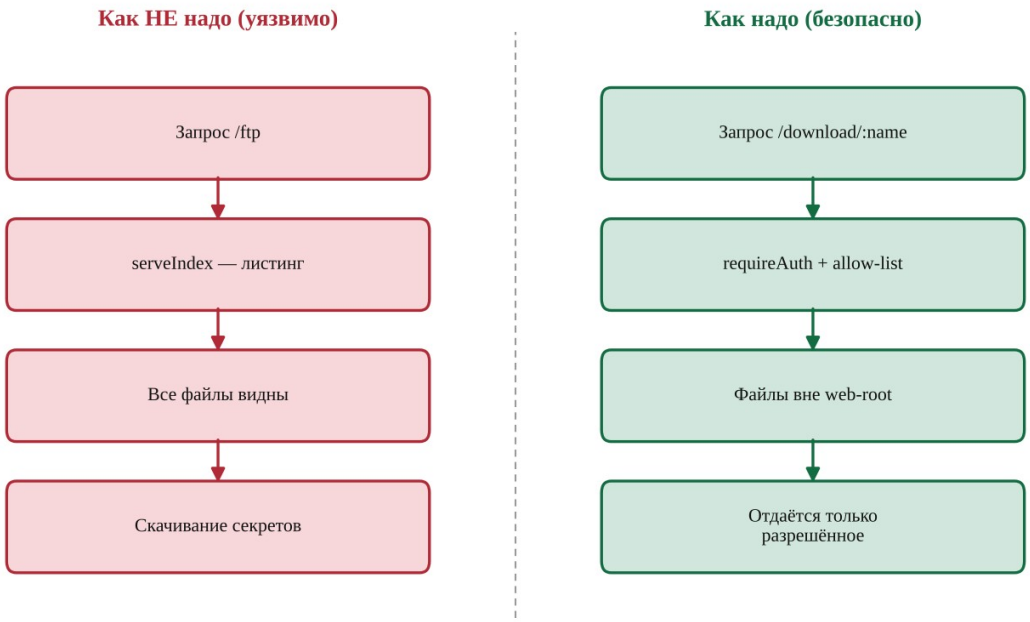


Рисунок 13 — Уязвимый и безопасный поток обработки (Раскрытие данных: доступ к каталогу /ftp)

Было (уязвимо):

```
// УЯЗВИМО: каталог раздаётся статикой с листингом
app.use('/ftp', serveIndex('ftp', { icons: true }))
app.use('/ftp', express.static('ftp'))
```

Стало (безопасно):

```
// БЕЗОПАСНО: без листинга, файлы вне веб-корня, по авторизации
// служебные файлы перемещены в /var/app/private (вне public)
app.get('/download/:name', requireAuth, (req, res) => {
  const name = path.basename(req.params.name)      // срезаем путь
  const allowed = ['invoice.pdf', 'terms.pdf']
  if (!allowed.includes(name)) return res.status(404).end()
  res.sendFile(path.join('/var/app/private', name))
})
```

Уязвимость №14. Обход фильтра расширений через Poison Null Byte (path traversal)

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Высокая.

3.14.1. Как нашли

Доступ к /ftp ограничен белым списком расширений (.md, .pdf). Однако фильтр реализован неверно и обходится «отравленным нулевым байтом» (poison null byte), что открывает доступ к файлам с другими расширениями.

3.14.2. Как использовать

Для скачивания запрещённого файла (например .bak или .conf) к имени добавляется закодированный нулевой байт и разрешённое расширение:

```
http://<IP>:3000/ftp/package.json.bak%2500.md
```

Сервер проверяет «.md», но ОС обрезает путь на нулевом байте и отдаёт package.json.bak, раскрывая зависимости и секреты.

3.14.3. Как исправить

Рекомендуемая модель: Нормализация пути + canonical-проверка внутри каталога + запрет %00.

Проверка расширения по «хвосту» строки обходится нулевым байтом. Модель: сначала декодировать и нормализовать путь, отклонять любые управляющие символы (в т.ч. %00), затем сверять канонический абсолютный путь — он должен лежать строго внутри разрешённого каталога (defense-in-depth), а не полагаться на проверку суффикса.

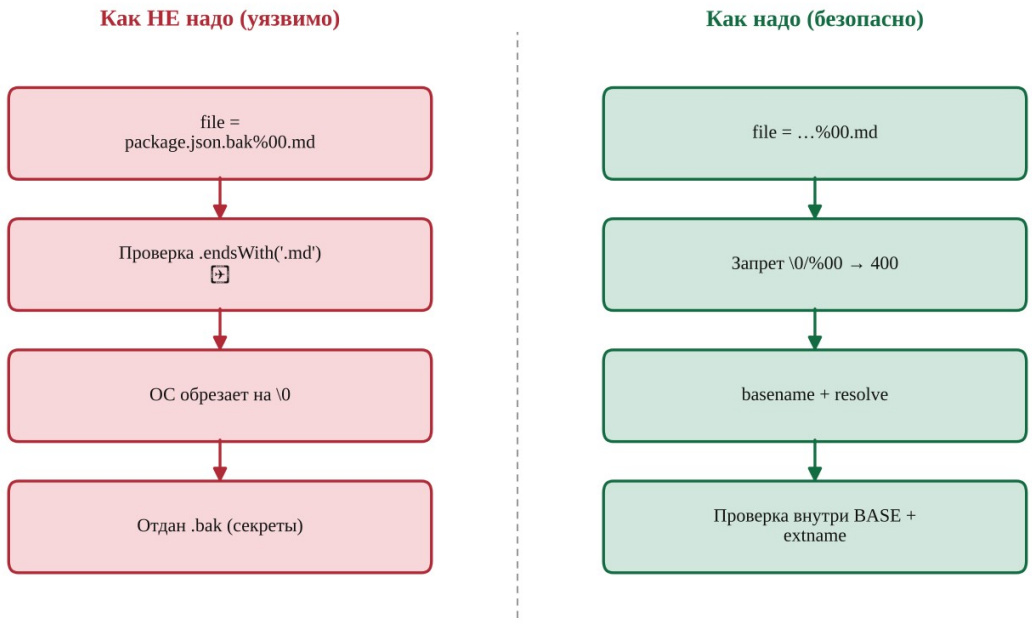


Рисунок 14 — Уязвимый и безопасный поток обработки (Обход фильтра расширений через Poison Null Byte (path traversal))

Было (уязвимо):

```
// УЯЗВИМО: проверка суффикса, уязвимая к %00
app.get('/ftp/:file', (req, res) => {
  const f = req.params.file
  if (f.endsWith('.md') || f.endsWith('.pdf'))   // 'x.bak\0.md' пройдёт
    res.sendFile(path.join('ftp', f))
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: запрет \0, нормализация, canonical-проверка
const BASE = path.resolve('ftp')
app.get('/ftp/:file', (req, res) => {
  const raw = req.params.file
  if (raw.includes('\0') || raw.includes('%00'))
    return res.status(400).end()
  const name = path.basename(raw)                  // убираем ../ и пути
  const full = path.resolve(BASE, name)
  if (!full.startsWith(BASE + path.sep))           // строго внутри BASE
    return res.status(400).end()
  if (!['.md', '.pdf'].includes(path.extname(full)))
    return res.status(404).end()
  res.sendFile(full)
})
```

Уязвимость №15. Раскрытие конфиденциального документа о поглощениях

Категория OWASP: A01:2021 – Broken Access Control. **Критичность:** Средняя.

3.15.1. Как нашли

В каталоге /ftp присутствует файл acquisitions.md — внутренний документ о планируемых поглощениях, не предназначенный для публичного доступа.

3.15.2. Как использовать

Напрямую скачивается http://<IP>:3000/ftp/acquisitions.md. Документ открывается без авторизации и содержит коммерческую тайну — демонстрация утечки чувствительных данных.

3.15.3. Как исправить

Рекомендуемая модель: Вынос конфиденциальных документов из публичного доступа + авторизация.

Внутренний документ лежал в публичном /ftp. Модель: конфиденциальные файлы хранить вне веб-корня, доступ — только авторизованным ролям, с журналированием обращений.

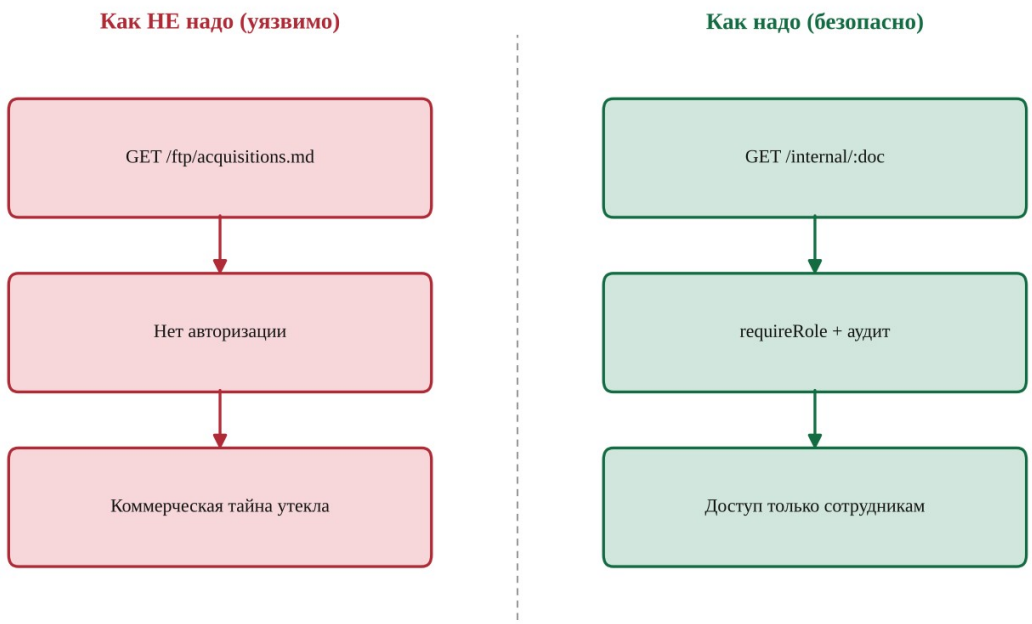


Рисунок 15 — Уязвимый и безопасный поток обработки (Раскрытие конфиденциального документа о поглощениях)

Было (уязвимо):

```
// УЯЗВИМО: внутренний документ публично доступен
// GET /ftp/acquisitions.md  → 200 OK без авторизации
```

Стало (безопасно):

```
// БЕЗОПАСНО: только для роли, с аудитом
app.get('/internal/:doc', requireRole('employee'), (req, res) => {
  audit.log(req.user.id, 'read', req.params.doc)   // журналирование
  res.sendFile(path.join('/var/app/confidential',
                         path.basename(req.params.doc)))
})
```

Уязвимость №16. Доступ к журналам сервера (/support/logs)

Категория OWASP: A09:2021 – Security Logging and Monitoring Failures.

Критичность: Средняя.

3.16.1. Как нашли

Эндпоинт /support/logs отдаёт журналы доступа сервера без аутентификации. В логах могут содержаться токены, пути и иные чувствительные сведения.

3.16.2. Как использовать

Открывается http://<IP>:3000/support/logs — отображается содержимое лог-файлов, из которых можно извлечь служебную информацию о запросах и пользователях.

3.16.3. Как исправить

Рекомендуемая модель: Закрытие журналов авторизацией + отказ от логирования секретов.

Логи доступны без авторизации и содержат чувствительные данные. Модель: ограничить доступ к логам (роль/сеть), не логировать токены и пароли (маскирование), хранить журналы в защищённом хранилище.

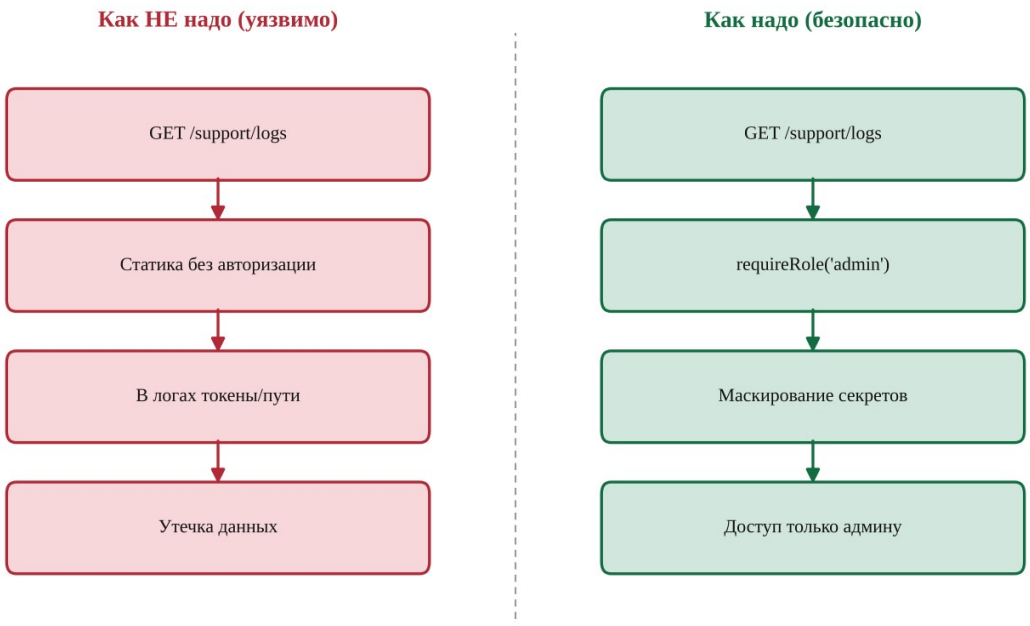


Рисунок 16 — Уязвимый и безопасный поток обработки (Доступ к журналам сервера (/support/logs))

Было (уязвимо):

```
// УЯЗВИМО: логи раздаются всем
app.use('/support/logs', express.static('logs'))
```

Стало (безопасно):

```
// БЕЗОПАСНО: по роли + маскирование секретов в логах
app.get('/support/logs', requireRole('admin'), (req, res) => {
  res.sendFile(path.resolve('logs/access.log'))
})
// логгер маскирует токены:
morgan((tokens, req) => line.replace(/Bearer\s+[\w.-]+/g, 'Bearer ***'))
```

Уязвимость №17. XXE: чтение файлов через загрузку XML

Категория OWASP: A05:2021 – Security Misconfiguration (XXE).

Критичность: Критическая.

3.17.1. Как нашли

Функция обработки жалоб (complaint) допускает загрузку файлов, включая XML. Парсер настроен с обработкой внешних сущностей (external entities), что открывает XXE.

3.17.2. Как использовать

Готовится XML-файл с объявлением внешней сущности, ссылающейся на локальный файл, и загружается через /file-upload:

```
<?xml version="1.0"?>
<!DOCTYPE foo [ <!ENTITY xxe SYSTEM "file:///etc/passwd"> ]>
<foo>&xxe;</foo>
```

В ответе раскрывается содержимое /etc/passwd (на Windows-стенде — соответствующий системный файл, например file:///c:/windows/system.ini).

3.17.3. Как исправить

Рекомендуемая модель: Отключение DTD и внешних сущностей в XML-парсере.

XXE возникает из-за обработки внешних сущностей XML-парсером. Модель: полностью отключить DTD и внешние сущности (noent=false, запрет DOCTYPE), предпочесть форматы без сущностей (JSON), ограничить тип/размер загрузок и запускать парсинг с минимальными привилегиями.

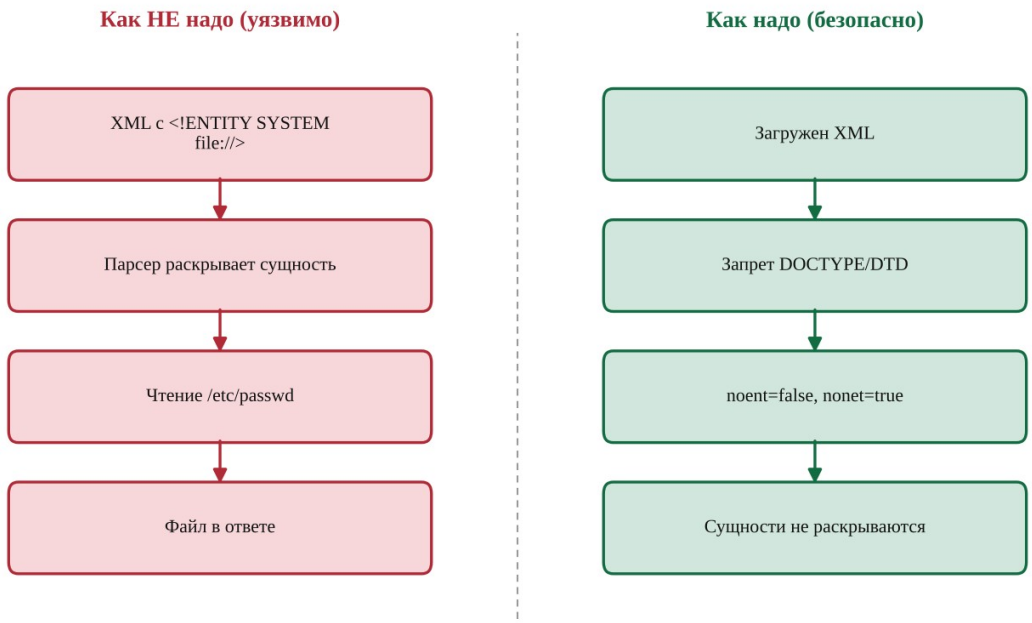


Рисунок 17 — Уязвимый и безопасный поток обработки (XXE: чтение файлов через загрузку XML)

Было (уязвимо):

```
// УЯЗВИМО: внешние сущности разрешены
const libxml = require('libxmljs2')
const doc = libxml.parseXml(fileContent, {
  noent: true,   // раскрывать сущности
  dtdload: true  // загружать внешние DTD → file:///etc/passwd
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: DTD/сущности отключены
const libxml = require('libxmljs2')
```

```
if (/<!DOCTYPE/i.test(fileContent))            // запрет DOCTYPE
  throw new Error('DTD is not allowed')
const doc = libxml.parseXml(fileContent, {
  noent: false,     // не раскрывать сущности
  dtdload: false,   // не загружать внешние DTD
  nonet: true       // запрет сетевых обращений
})
// предпочтительно: принимать JSON вместо XML
```

Уязвимость №18. SSRF через подмену URL профильного изображения

Категория OWASP: A10:2021 – Server-Side Request Forgery. **Критичность:** Высокая.

3.18.1. Как нашли

Профиль пользователя позволяет задать URL изображения, который сервер самостоятельно загружает (POST /profile/image/url). Сервер делает запрос по произвольному адресу — признак SSRF.

3.18.2. Как использовать

В поле URL аватара указывается внутренний адрес, недоступный снаружи, например http://localhost:3000/... или адрес служебного сервиса. Сервер выполняет запрос от своего имени, что позволяет сканировать внутреннюю сеть и обращаться к закрытым ресурсам.

3.18.3. Как исправить

Рекомендуемая модель: Allow-list доменов + запрет приватных сетей + отказ возвращать ответ.

SSRF: сервер ходит по URL, заданному пользователем. Модель: разрешать только белый список доменов, резолвить DNS и запрещать приватные диапазоны (127/8, 10/8, 172.16/12, 192.168/16, 169.254/16), не возвращать тело внутреннего ответа клиенту, использовать отдельный egress-прокси и таймауты.

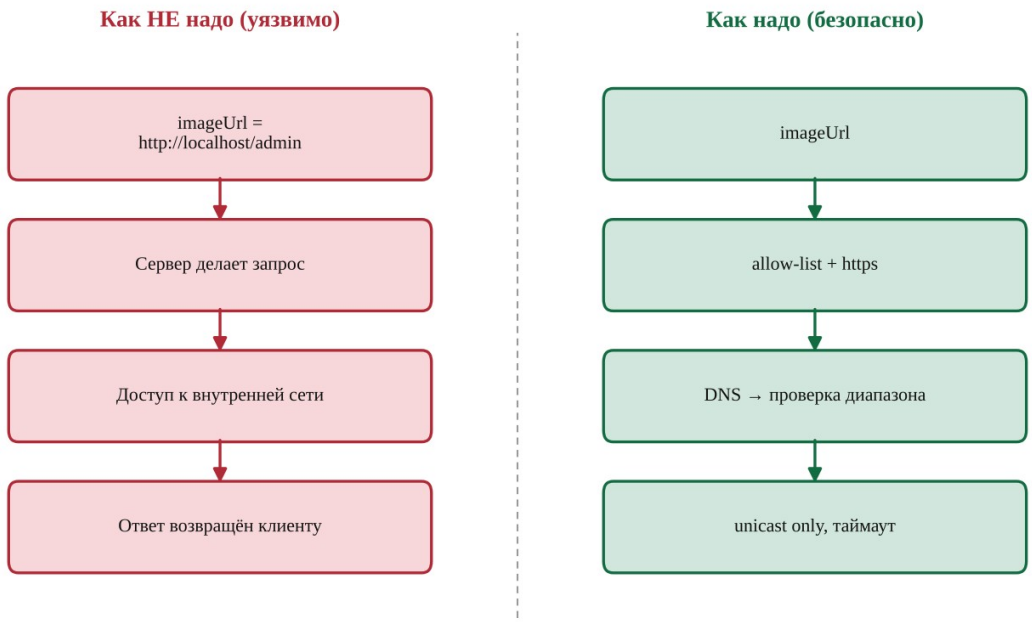


Рисунок 18 — Уязвимый и безопасный поток обработки (SSRF через подмену URL профильного изображения)

Было (уязвимо):

```
// УЯЗВИМО: сервер качает произвольный URL
app.post('/profile/image/url', async (req, res) => {
  const img = await fetch(req.body.imageUrl)  // http://localhost:…
пройдёт
  saveAvatar(await img.buffer())
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: allow-list + запрет приватных адресов
import dns from 'dns/promises'; import ipaddr from 'ipaddr.js'
const ALLOWED = ['images.example.com']
app.post('/profile/image/url', async (req, res) => {
  const url = new URL(req.body.imageUrl)
  if (url.protocol !== 'https:' || !ALLOWED.includes(url.hostname))
    return res.status(400).end()
  const { address } = (await dns.lookup(url.hostname))
  const range = ipaddr.parse(address).range()
  if (range !== 'unicast')            // отсекаем
loopback/private/linkLocal
    return res.status(400).end()
  const img = await fetch(url, { timeout: 3000 })
  saveAvatar(await img.buffer())
})
```

Уязвимость №19. Подделка JWT через алгоритм none / слабый секрет

Категория OWASP: A02:2021 – Cryptographic Failures. **Критичность:** Критическая.

3.19.1. Как нашли

Сессия использует JWT, хранящийся в localStorage. При анализе на jwt.io видно, что заголовок задаёт алгоритм, а в коде проверка подписи реализована нестрого, что допускает подделку (alg:none и/или слабый общий ключ).

3.19.2. Как использовать

Payload токена изменяется (например email администратора), заголовок переключается на alg:none, подпись удаляется — если сервер принимает такой токен, достигается повышение привилегий. Альтернативно подпись пересчитывается при известном/слабом секрете.

3.19.3. Как исправить

Рекомендуемая модель: Жёсткая фиксация алгоритма (RS256), запрет alg:none, защита ключа.

Подделка JWT возможна при приёме alg:none или слабом общем секрете. Модель: проверять подпись строго с фиксированным списком алгоритмов (algorithms:['RS256']), никогда не доверять полю alg из токена, хранить приватный ключ в секрет-хранилище, задавать короткий срок жизни и ротацию ключей.

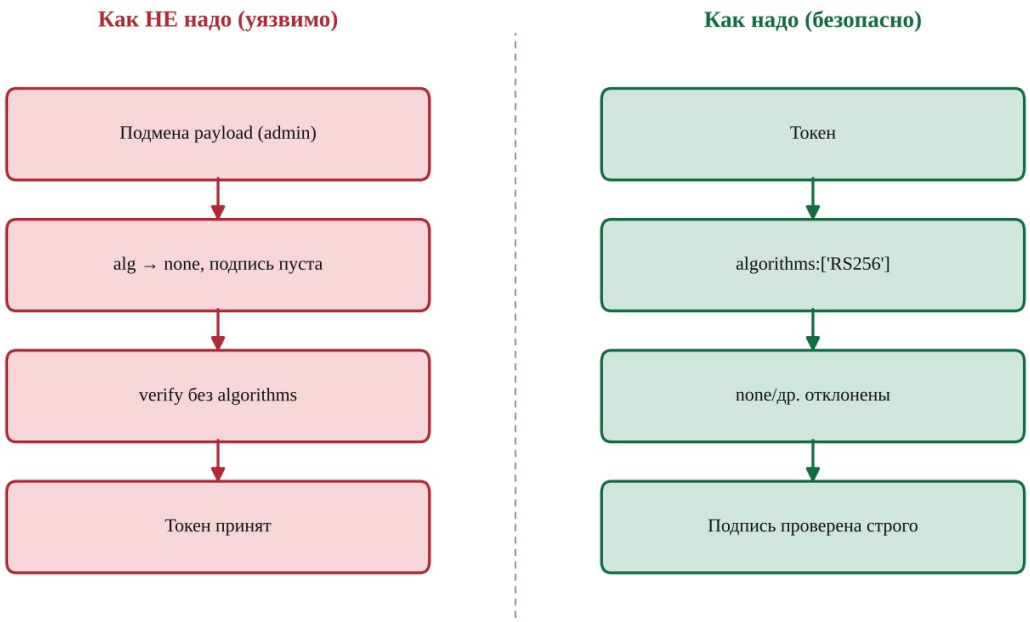


Рисунок 19 — Уязвимый и безопасный поток обработки (Подделка JWT через алгоритм none / слабый секрет)

Было (уязвимо):

```
// УЯЗВИМО: алгоритм берётся из токена, none проходит
jwt.verify(token, publicKey)   // без ограничения алгоритмов
// заголовок {"alg":"none"} → подпись не проверяется
```

Стало (безопасно):

```
// БЕЗОПАСНО: только RS256, none запрещён
jwt.verify(token, publicKey, {
  algorithms: ['RS256'],       // фиксируем алгоритм
  maxAge: '15m'                // короткий срок жизни
})
// секрет/приватный ключ — в KMS/Vault, регулярная ротация
```

Уязвимость №20. Слабое хеширование паролей (MD5)

Категория OWASP: A02:2021 – Cryptographic Failures. **Критичность:** Высокая.

3.20.1. Как нашли

После выгрузки таблицы Users (через UNION-инъекцию) видно, что пароли хранятся как MD5-хеши без соли — быстрый и устаревший алгоритм, непригодный для паролей.

3.20.2. Как использовать

Полученные хеши восстанавливаются офлайн: по радужным таблицам или через hashcat:

```
hashcat -m 0 -a 0 hashes.txt rockyou.txt
```

Несолёный MD5 ломается за секунды для распространённых паролей, раскрывая учётные данные пользователей.

3.20.3. Как исправить

Рекомендуемая модель: Адаптивное хеширование с солью (bcrypt/argon2).

Пароли хранились как несолёный MD5 — мгновенно ломается. Модель: хранить пароли через медленные адаптивные функции с индивидуальной солью (argon2id или bcrypt с cost≥12), миграция — перехешировать при первом успешном входе; MD5/SHA-1 для паролей запрещены.

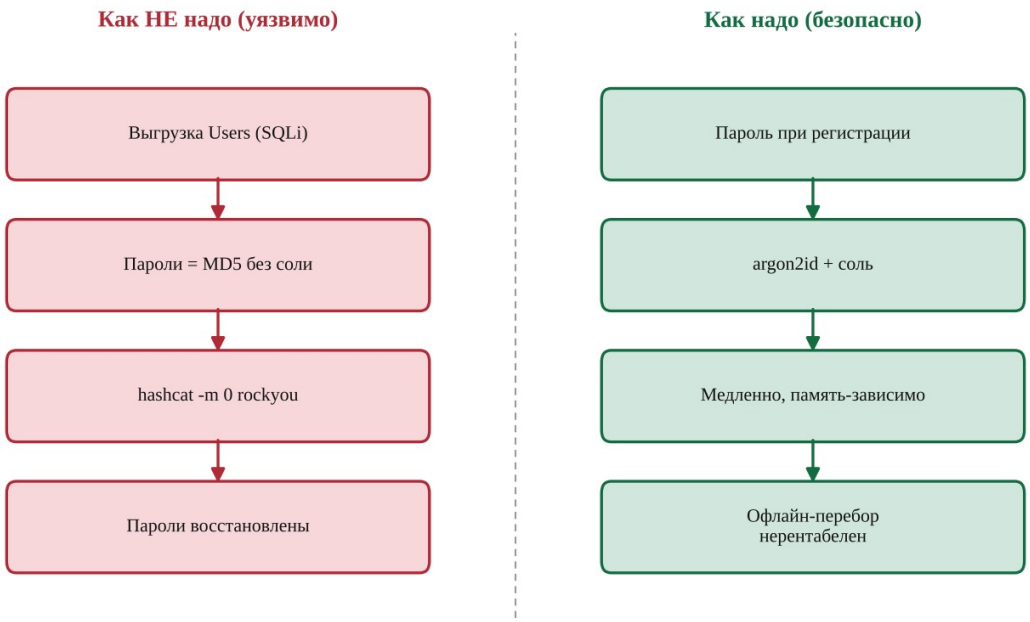


Рисунок 20 — Уязвимый и безопасный поток обработки (Слабое хеширование паролей (MD5))

Было (уязвимо):

```
// УЯЗВИМО: несолёный MD5
const hash = crypto.createHash('md5')
  .update(password).digest('hex')   // ломается hashcat за секунды
```

Стало (безопасно):

```
// БЕЗОПАСНО: argon2id (или bcrypt)
import argon2 from 'argon2'
const hash = await argon2.hash(password, {
  type: argon2.argon2id, memoryCost: 19456, timeCost: 2
})
// проверка:
const ok = await argon2.verify(user.password, inputPassword)
// bcrypt-альтернатива: await bcrypt.hash(password, 12)
```

Уязвимость №21. Небезопасная десериализация / выполнение кода

Категория OWASP: A08:2021 – Software and Data Integrity Failures.

Критичность: Критическая.

3.21.1. Как нашли

Приложение использует сторонние пакеты, один из которых небезопасно обрабатывает пользовательские данные (десериализация/вычисление выражений). Это создаёт риск выполнения произвольного кода на сервере.

3.21.2. Как использовать

Формируется полезная нагрузка, которую уязвимая библиотека вычисляет на сервере (например объект, приводящий к выполнению кода при десериализации). Отправка нагрузки в соответствующий эндпоинт приводит к исполнению команд или зависанию процесса (DoS).

3.21.3. Как исправить

Рекомендуемая модель: Запрет небезопасной десериализации + валидация + обновление зависимостей + изоляция.

Небезопасная обработка данных уязвимой библиотекой ведёт к RCE/DoS. Модель: не десериализовать недоверенные данные опасными методами, принимать только данные по строгой схеме (JSON Schema), обновлять зависимости (устранять CVE), таймауты и лимиты на вычисления, запуск процесса в изоляции с минимальными правами.

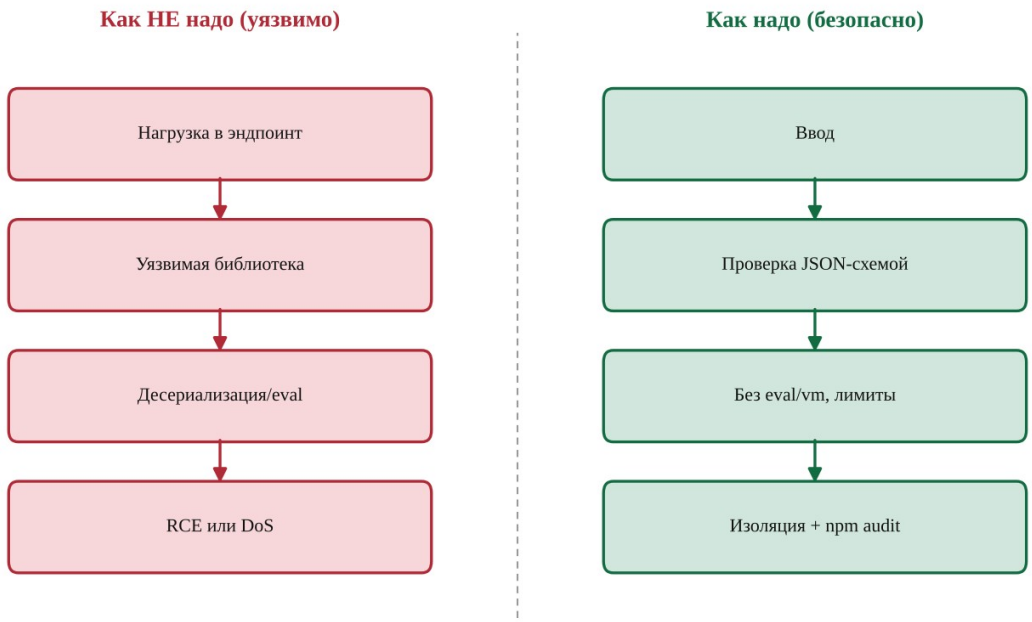


Рисунок 21 — Уязвимый и безопасный поток обработки (Небезопасная десериализация / выполнение кода)

Было (уязвимо):

```
// УЯЗВИМО: вычисление/десериализация недоверенного ввода
const vm = require('vm')
vm.runInNewContext(req.body.expr)   // произвольный код/зависание
```

Стало (безопасно):

```
// БЕЗОПАСНО: схема + безопасный парсер + лимиты
import Ajv from 'ajv'
const ajv = new Ajv()
const validate = ajv.compile(schema)      // строгая JSON-схема
if (!validate(req.body)) return res.status(400).end()
// никакого eval/vm над вводом; тяжёлые операции — в изолированном
// воркере с таймаутом и ограничением памяти; npm audit в CI
```

Уязвимость №22. Уязвимые и устаревшие компоненты (перезапись legal.md)

Категория OWASP: A06:2021 – Vulnerable and Outdated Components.

Критичность: Высокая.

3.22.1. Как нашли

В package.json присутствуют устаревшие зависимости с известными CVE. Одна из них (обработка ZIP/распаковка) позволяет выход за пределы каталога при загрузке архива.

3.22.2. Как использовать

Готовится ZIP-архив, где путь файла содержит переход вверх по дереву (../../ftp/legal.md), и загружается через функцию импорта/загрузки. При распаковке файл записывается поверх легитимного legal.md (zip slip / path traversal).

3.22.3. Как исправить

Рекомендуемая модель: Актуализация зависимостей (SCA) + безопасная распаковка архивов.

Устаревшая зависимость распаковки позволяет zip slip (path traversal). Модель: регулярно обновлять зависимости и сканировать их (npm audit, Dependabot, Snyk), при распаковке нормализовать и проверять путь каждого элемента, запрещать переходы ../ и абсолютные пути, ограничивать каталог назначения и размер.

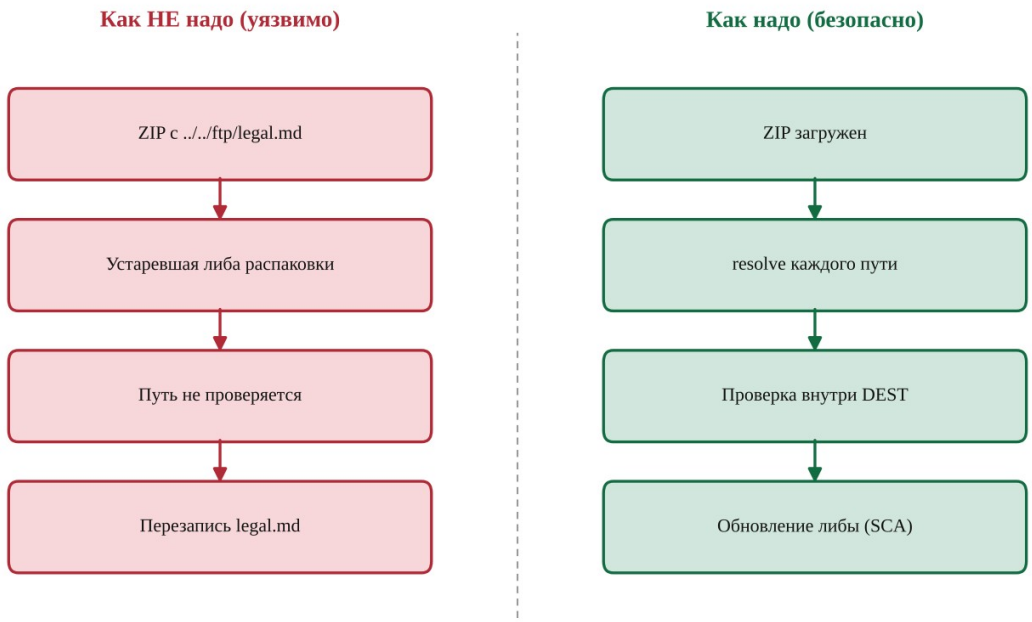


Рисунок 22 — Уязвимый и безопасный поток обработки (Уязвимые и устаревшие компоненты (перезапись legal.md))

Было (уязвимо):

```
// УЯЗВИМО: имя файла из архива используется как путь
zip.getEntries().forEach(e => {
  fs.writeFileSync(path.join('uploads', e.entryName), e.getData())
  // entryName = '../../ftp/legal.md' → перезапись
})
```

Стало (безопасно):

```
// БЕЗОПАСНО: проверка каждого пути (anti zip-slip)
const DEST = path.resolve('uploads')
zip.getEntries().forEach(e => {
  const target = path.resolve(DEST, e.entryName)
  if (!target.startsWith(DEST + path.sep))        // внутри DEST?
    throw new Error('Path traversal blocked: ' + e.entryName)
  fs.writeFileSync(target, e.getData())
})
// + npm audit fix / Dependabot для устранения CVE библиотеки
```

Уязвимость №23. Обход защиты от автоматизации (10+ отзывов за 10 секунд)

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Средняя.

3.23.1. Как нашли

Форма обратной связи (feedback) не имеет серверного ограничения частоты: CAPTCHA проверяется недостаточно строго, и можно отправить множество запросов подряд.

3.23.2. Как использовать

Скриптом (например на Python requests в Kali) отправляется 10 и более отзывов за 10 секунд на /api/Feedbacks, при необходимости повторно используя валидный ответ CAPTCHA. Защита от автоматизации обходится.

3.23.3. Как исправить

Рекомендуемая модель: Серверный rate-limit + проверяемая одноразовая CAPTCHA + антибот.

Форма не ограничивает частоту на сервере. Модель: серверное ограничение частоты отправок, одноразовая CAPTCHA, проверяемая на сервере (каждый ответ валиден однократно), обнаружение аномалий/ботов, идемпотентность отправки.

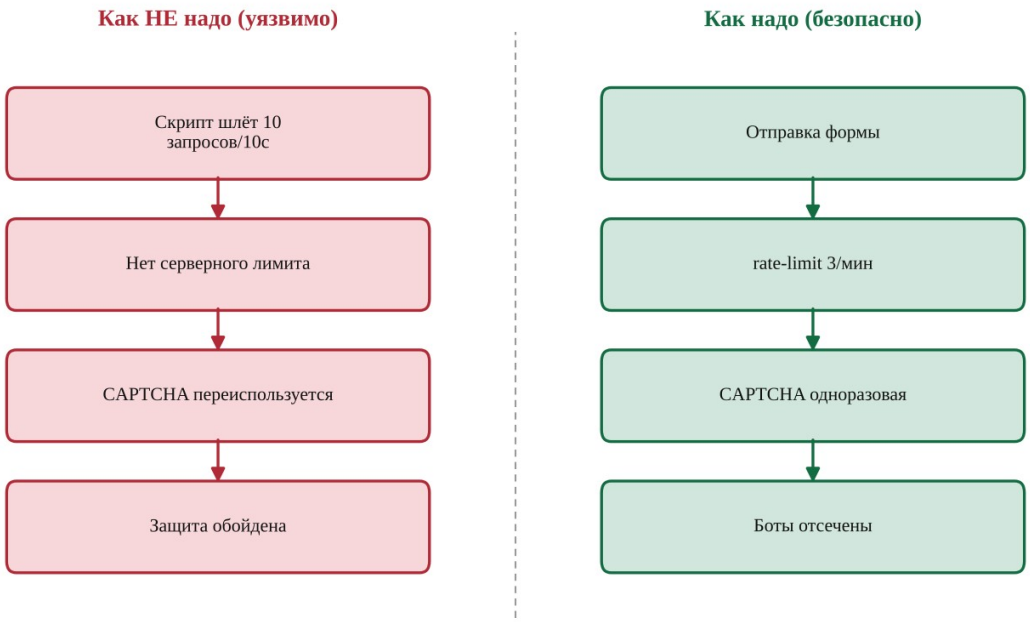


Рисунок 23 — Уязвимый и безопасный поток обработки (Обход защиты от автоматизации (10+ отзывов за 10 секунд))

Было (уязвимо):

```
// УЯЗВИМО: нет серверного лимита, CAPTCHA переиспользуется
app.post('/api/Feedbacks', (req, res) => saveFeedback(req.body))
```

Стало (безопасно):

```
// БЕЗОПАСНО: rate-limit + одноразовая CAPTCHA
import rateLimit from 'express-rate-limit'
const fbLimiter = rateLimit({ windowMs: 60*1000, max: 3 })
app.post('/api/Feedbacks', fbLimiter, async (req, res) => {
  const c = await db.captchas.findOne({ id: req.body.captchaId })
  if (!c || c.used || c.answer !== Number(req.body.captcha))
    return res.status(400).json({ error: 'Captcha invalid' })
  await db.captchas.update({ id: c.id }, { used: true }) // одноразовость
  saveFeedback(req.body)
})
```

Уязвимость №24. Отсутствие/слабость заголовков безопасности (CSP)

Категория OWASP: A05:2021 – Security Misconfiguration. **Критичность:** Средняя.

3.24.1. Как нашли

Анализ заголовков ответа (например через curl -I или вкладку Network) показывает отсутствие либо слабость Content-Security-Policy и других security-заголовков (X-Frame-Options, X-Content-Type-Options), что усиливает влияние XSS и clickjacking.

3.24.2. Как использовать

Так как CSP не запрещает инлайновые скрипты и небезопасные источники, ранее показанные XSS-нагрузки исполняются без ограничений. Отсутствие X-Frame-Options допускает встраивание сайта в iframe (clickjacking).

3.24.3. Как исправить

Рекомендуемая модель: helmet: строгая CSP + security-заголовки.

Слабая/отсутствующая CSP усиливает XSS и допускает clickjacking. Модель: задать строгую Content-Security-Policy без unsafe-inline (скрипты — только со своего источника/по nonce), а также X-Frame-Options, X-Content-Type-Options, Referrer-Policy и HSTS; в Express — через helmet.

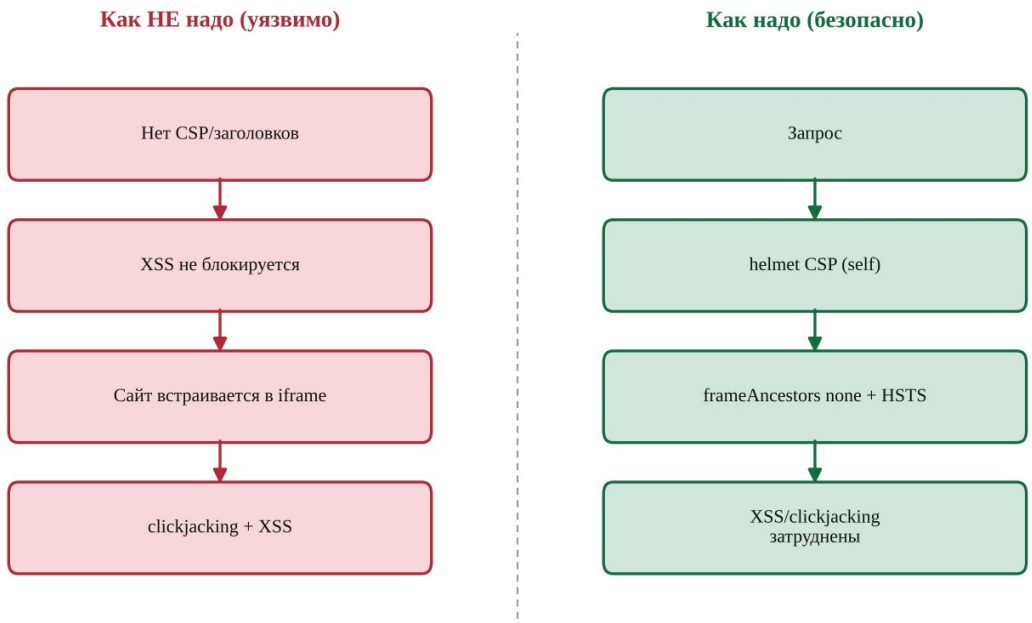


Рисунок 24 — Уязвимый и безопасный поток обработки (Отсутствие/слабость заголовков безопасности (CSP))

Было (уязвимо):

```
// УЯЗВИМО: заголовки безопасности не заданы
const app = express()   // нет CSP, X-Frame-Options, nosniff
```

Стало (безопасно):

```
// БЕЗОПАСНО: helmet со строгой CSP
import helmet from 'helmet'
app.use(helmet({
  contentSecurityPolicy: {
    directives: {
      defaultSrc: ["'self'"],
      scriptSrc:  ["'self'"],         // без unsafe-inline
      objectSrc:  ["'none'"],
      frameAncestors: ["'none'"]      // анти-clickjacking
    }
  },
  hsts: { maxAge: 31536000 }
}))
```

Уязвимость №25. Слабая криптография купонов (предсказуемые скидки)

Категория OWASP: A02:2021 – Cryptographic Failures. **Критичность:** Низкая.

3.25.1. Как нашли

Коды купонов формируются по слабому обратимому алгоритму (кодирование вместо криптостойкой генерации). Изучив несколько валидных кодов, можно восстановить закономерность.

3.25.2. Как использовать

По вскрытой схеме генерируется собственный валидный купон с крупной скидкой и применяется в корзине, что приводит к финансовым потерям магазина.

3.25.3. Как исправить

Рекомендуемая модель: Серверная генерация купонов криптослучайно + хранение и проверка по БД.

Купоны формировались обратимым кодированием — восстанавливаются по образцам. Модель: генерировать купоны криптослучайно, хранить в БД, при применении проверять по хранилищу (а не вычислять из кода), ограничивать срок действия и число использований.

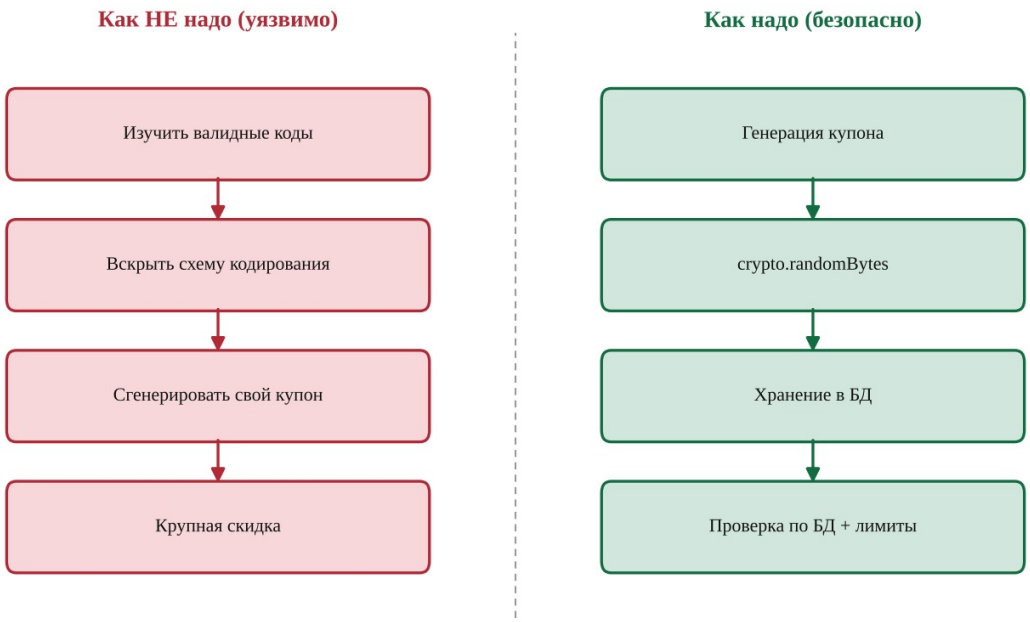


Рисунок 25 — Уязвимый и безопасный поток обработки (Слабая криптография купонов (предсказуемые скидки))

Было (уязвимо):

```
// УЯЗВИМО: код купона вычисляется из даты (обратимо)
const coupon = base85(z85encode(month + '-' + discount))
// зная схему, можно сгенерировать свой код скидки
```

Стало (безопасно):

```
// БЕЗОПАСНО: случайный код, проверка по БД
import crypto from 'crypto'
const code = crypto.randomBytes(9).toString('base64url') // непредсказуем
await db.coupons.insert({ code, discount: 10,
  expiresAt: Date.now()+7*864e5, maxUses: 100, used: 0 })

// применение:
const c = await db.coupons.findOne({ code: req.body.code })
if (!c || c.used >= c.maxUses || c.expiresAt < Date.now())
  return res.status(400).json({ error: 'Invalid coupon' })
```

Уязвимость №26. Безопасность через сокрытие (утечка до анонса)

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Низкая.

3.26.1. Как нашли

Некоторые ресурсы (например страница о предстоящей продаже токенов) считаются «скрытыми», но доступны по прямой ссылке/в бандле до официального анонса. Защита построена на сокрытии, а не на контроле доступа.

3.26.2. Как использовать

Через анализ main.js или перебор путей находится неанонсированный ресурс и открывается напрямую, раскрывая информацию раньше срока.

3.26.3. Как исправить

Рекомендуемая модель: Реальный контроль доступа вместо сокрытия (feature flag + авторизация).

Ресурс считался «скрытым», но доступен по прямой ссылке. Модель: защищать закрытый контент авторизацией и серверными feature-флагами по времени, а не отсутствием ссылки; до наступления условия сервер отдаёт 404/403.

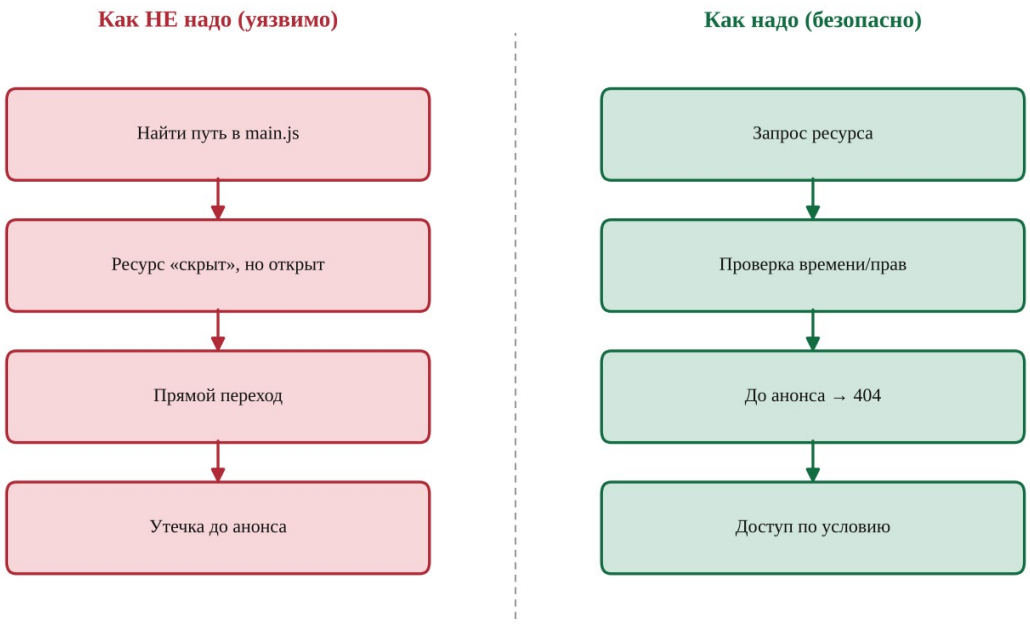


Рисунок 26 — Уязвимый и безопасный поток обработки (Безопасность через сокрытие (утечка до анонса))

Было (уязвимо):

```
// УЯЗВИМО: доступ есть, просто нет ссылки в меню
app.get('/tokensale', (req, res) => res.render('tokensale'))
```

Стало (безопасно):

```
// БЕЗОПАСНО: серверная проверка времени/прав
const LAUNCH = Date.parse('2026-12-01T00:00:00Z')
app.get('/tokensale', (req, res) => {
  if (Date.now() < LAUNCH && !isInsider(req.user))
    return res.status(404).end()      // до анонса — недоступно
  res.render('tokensale')
})
```

Уязвимость №27. Небезопасная загрузка файлов (тип и размер)

Категория OWASP: A04:2021 – Insecure Design. **Критичность:** Средняя.

3.27.1. Как нашли

Функция загрузки (жалобы) проверяет тип файла в основном на клиенте и слабо ограничивает размер. Серверная валидация недостаточна, что позволяет загружать неожиданные типы файлов.

3.27.2. Как использовать

Через перехват запроса загружается файл запрещённого типа или превышающий заявленный лимит (например крупный файл или не-PDF вместо PDF). Это может приводить к обработке опасного содержимого (связка с XXE) или к DoS.

3.27.3. Как исправить

Рекомендуемая модель: Серверная проверка типа (magic bytes) и размера + хранение вне web-root.

Тип/размер проверялись слабо (в основном на клиенте). Модель: валидировать на сервере тип по сигнатуре содержимого (magic bytes), ограничивать размер, использовать белый список типов, хранить загрузки вне веб-корня без прав на исполнение, при необходимости — антивирусная проверка.

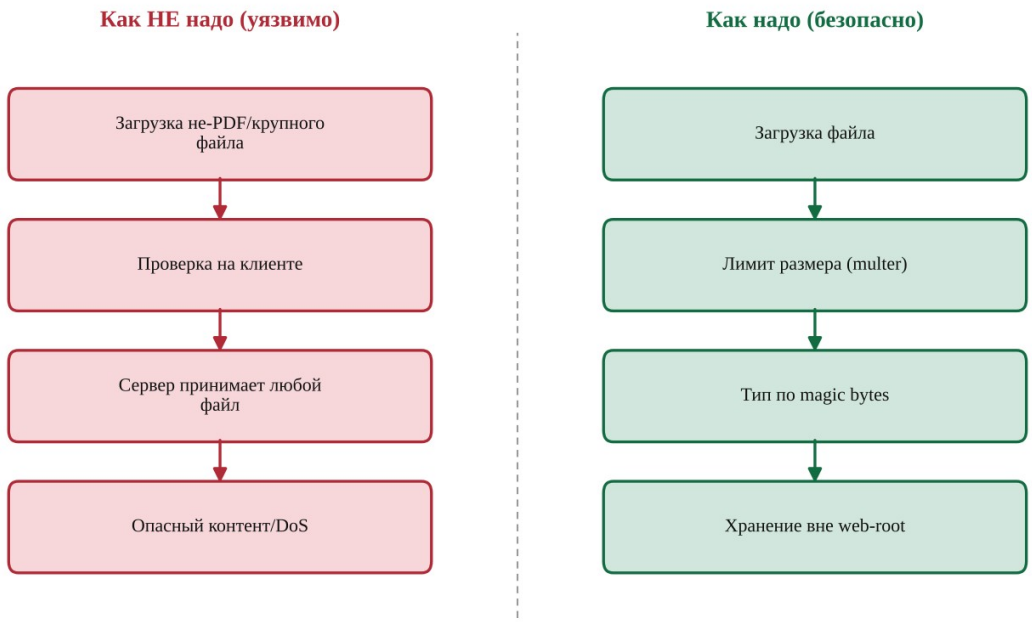


Рисунок 27 — Уязвимый и безопасный поток обработки (Небезопасная загрузка файлов (тип и размер))

Было (уязвимо):

```
// УЯЗВИМО: доверие к расширению/клиенту, без лимита
const upload = multer({ dest: 'uploads/' })
app.post('/file-upload', upload.single('file'), handler) // любой
тип/размер
```

Стало (безопасно):

```
// БЕЗОПАСНО: лимит размера + проверка magic bytes
import { fileTypeFromBuffer } from 'file-type'
const upload = multer({
  storage: multer.memoryStorage(),
  limits: { fileSize: 1024 * 1024 }          // 1 МБ
})
app.post('/file-upload', upload.single('file'), async (req, res) => {
  const ft = await fileTypeFromBuffer(req.file.buffer)
  const allowed = ['application/pdf', 'application/xml']
```

```
  if (!ft || !allowed.includes(ft.mime))      // тип по содержимому
    return res.status(415).json({ error: 'Unsupported type' })
  await saveOutsideWebroot(req.file.buffer)   // вне public, без +x
})
```

4. Заключение

В ходе работы на стенде OWASP Juice Shop обнаружено и проэксплуатировано 27 уязвимостей, охватывающих все десять категорий OWASP Top 10 2021: внедрение (SQL/NoSQL/XSS), нарушение контроля доступа (IDOR, mass assignment, доступ к админке), криптографические недостатки (слабое хеширование, подделка JWT), ошибки конфигурации (открытый /ftp, XXE, отсутствие security-заголовков), уязвимые компоненты, SSRF, небезопасный дизайн и недостатки аутентификации.

Для каждой уязвимости показаны обнаружение, эксплуатация и устранение с конкретной моделью, кодом и схемой. Большинство проблем устраняется базовыми практиками безопасной разработки: параметризация и валидация ввода, экранирование вывода, серверный контроль доступа по ролям, безопасное хранение секретов и паролей, актуализация зависимостей и корректная конфигурация security-заголовков. Результаты подтверждают ценность OWASP Juice Shop как площадки для отработки навыков наступательной и оборонительной безопасности.

5. Список литературы

Формат: Электронный ресурс [URL] — название статьи, автор, дата обращения. Номера соответствуют надстрочным сноскам в тексте.

1. Электронный ресурс [https://owasp.org/www-project-juice-shop/] — OWASP Juice Shop Project, OWASP Foundation, дата обращения: 15.09.2026.

2. Электронный ресурс [https://pwning.owasp-juice.shop/] — Pwning OWASP Juice Shop (companion guide), Björn Kimminich, дата обращения: 15.09.2026.

3. Электронный ресурс [https://owasp.org/www-project-top-ten/] — OWASP Top 10, OWASP Foundation, дата обращения: 16.09.2026.

4. Электронный ресурс [https://owasp.org/Top10/A03_2021-Injection/] — A03:2021 – Injection, OWASP Foundation, дата обращения: 15.09.2026.

5. Электронный ресурс [https://portswigger.net/web-security/sql-injection/union-attacks] — SQL injection UNION attacks, PortSwigger Web Security Academy, дата обращения: 16.09.2026.

6. Электронный ресурс [https://owasp.org/www-community/attacks/NoSQL_injection] — NoSQL injection, OWASP Community, дата обращения: 18.09.2026.

7. Электронный ресурс [https://owasp.org/www-community/attacks/DOM_Based_XSS] — DOM Based XSS, OWASP Community, дата обращения: 19.09.2026.

8. Электронный ресурс [https://owasp.org/www-community/attacks/xss/] — Cross Site Scripting (XSS), OWASP Community, дата обращения: 20.09.2026.

9. Электронный ресурс [https://portswigger.net/web-security/cross-site-scripting/stored] — Stored cross-site scripting, PortSwigger Web Security Academy, дата обращения: 22.09.2026.

10. Электронный ресурс [https://owasp.org/Top10/A01_2021-Broken_Access_Control/] — A01:2021 – Broken Access Control, OWASP Foundation, дата обращения: 23.09.2026.

11. Электронный ресурс [https://portswigger.net/web-security/access-control/idor] — Insecure direct object references (IDOR), PortSwigger Web Security Academy, дата обращения: 24.09.2026.

12. Электронный ресурс [https://cwe.mitre.org/data/definitions/639.html] — CWE-639: Authorization Bypass Through User-Controlled Key, MITRE CWE, дата обращения: 25.09.2026.

13. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Mass_Assignment_Cheat_Sheet.html] — Mass Assignment Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 26.09.2026.

14. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Forgot_Password_Cheat_Sheet.html] — Forgot Password Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 27.09.2026.

15. Электронный ресурс [https://owasp.org/www-community/controls/Blocking_Brute_Force_Attacks] — Blocking Brute Force Attacks, OWASP Community, дата обращения: 28.09.2026.

16. Электронный ресурс [https://owasp.org/Top10/A05_2021-Security_Misconfiguration/] — A05:2021 – Security Misconfiguration, OWASP Foundation, дата обращения: 29.09.2026.

17. Электронный ресурс [https://owasp.org/www-community/attacks/Embedding_Null_Code] — Embedding Null Code (Poison Null Byte), OWASP Community, дата обращения: 30.09.2026.

18. Электронный ресурс [https://cwe.mitre.org/data/definitions/200.html] — CWE-200: Exposure of Sensitive Information, MITRE CWE, дата обращения: 01.10.2026.

19. Электронный ресурс [https://owasp.org/Top10/A09_2021-Security_Logging_and_Monitoring_Failures/] — A09:2021 – Security Logging and Monitoring Failures, OWASP Foundation, дата обращения: 02.10.2026.

20. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/XML_External_Entity_Prevention_Cheat_Sheet.html] — XML External Entity Prevention Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 03.10.2026.

21. Электронный ресурс [https://owasp.org/Top10/A10_2021-Server-Side_Request_Forgery_%28SSRF%29/] — A10:2021 – Server-Side Request Forgery (SSRF), OWASP Foundation, дата обращения: 05.10.2026.

22. Электронный ресурс [https://portswigger.net/web-security/jwt] — JWT attacks, PortSwigger Web Security Academy, дата обращения: 06.10.2026.

23. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Password_Storage_Cheat_Sheet.html] — Password Storage Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 07.10.2026.

24. Электронный ресурс [https://owasp.org/www-community/vulnerabilities/Deserialization_of_untrusted_data] — Deserialization of untrusted data, OWASP Community, дата обращения: 08.10.2026.

25. Электронный ресурс [https://owasp.org/Top10/A06_2021-Vulnerable_and_Outdated_Components/] — A06:2021 – Vulnerable and Outdated Components, OWASP Foundation, дата обращения: 09.10.2026.

26. Электронный ресурс [https://owasp.org/www-project-automated-threats-to-web-applications/] — OWASP Automated Threats to Web Applications, OWASP Foundation, дата обращения: 10.10.2026.
27. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/Content_Security_Policy_Cheat_Sheet.html] — Content Security Policy Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 11.10.2026.
28. Электронный ресурс [https://owasp.org/Top10/A02_2021-Cryptographic_Failures/] — A02:2021 – Cryptographic Failures, OWASP Foundation, дата обращения: 12.10.2026.
29. Электронный ресурс [https://cwe.mitre.org/data/definitions/656.html] — CWE-656: Reliance on Security Through Obscurity, MITRE CWE, дата обращения: 13.10.2026.
30. Электронный ресурс [https://cheatsheetseries.owasp.org/cheatsheets/File_Upload_Cheat_Sheet.html] — File Upload Cheat Sheet, OWASP Cheat Sheet Series, дата обращения: 14.10.2026.